Library User Manual

AXL library family
AXD

AJINEXTEK AXT

Product Information

Full information about other AJINEXTEK products
is available by visiting our Web Site at:
Home Page : www.ajinextek.com
E-Mail ::support@ajinextek.com

Useful Contact Information

Customer Support Seoul
Tel : 82-031-436-2180~2 Fax: 82-031-436-2183

Customer Support Cheonan
Tel : 82-041-555-9771~2 Fax: 82-041-555-9773

Customer Support Deagu
Tel : 82-053-593-3700~2 Fax: 82-053-593-3703

AJINEXTEK's sales team is always available to assist you in making your decision the final choice of boards or systems is solely and wholly theresponsibility of the buyer. AJINEXTEK's entire liability in respect of the board or systems is as set out in AJINEXTEK's standard terms and conditions of sale

ⓒ Copyright 2010. AJINEXTEK co.ltd. All rights reserved.

Content

CONTENT 3

HOW TO USE DIO LIBRARY 7

DIO Library Overview 7

Overview 7
Basic rules of DIO APIs 8
Description of DIO API group 10

Initialization and Exiting 11

Initialization of Library 11
Exiting Library 12

Setting Signal Level of Input/Output Port 15

Offset Description 15
API Description 17

Reading Input/Output Port 19

API Description 19

Writing Output Port 23

API Description 23

Interrupt 28

Setting Interrupt Rising / Falling Edge 28
Checking the Set Value for Interrupt Rising / Falling Edge 29
Setting the Use of Interrupt 30
Setting the Process Method for Interrupt 30

Advanced DIO API 37

Checking Signal Variance 37
Checking Signal Continuity 39
Keeping Signal for a Certain Period of Time 43
Signal Toggle 45

DIO COMMAND MANUAL INFORMATION 49

Header File 49

Terms for API 50

API Names in the Manual 50

Argument Names in the Manual 50

Interrupt Rising / Falling Edge Register 53

DIO COMMAND QUICK LIST 54

Board and Module 54

Interrupt 54

Checking the Setting of Input Level 55

Checking the Setting of Output Level 55

Reading Input Port 56

Reading and Writing Output Port 56

Advanced API 57

Define Sentence 58

DIO COMMAND FUNCTION LIST 59

Board and Module Information 59

AxdInfoIsDIOModule 60

AxdInfoGetModuleNo 62

AxdInfoGetModuleCount 64

AxdInfoGetInputCount 66

AxdInfoGetOutputCount 68

AxdInfoGetModule 70

Interrupt 72

AxdiInterruptSetModule 73

AxdiInterruptSetModuleEnable 75

AxdiInterruptGetModuleEnable 77

AxdiInterruptRead 79

AxdiInterruptEdgeSetBit 81

AxdiInterruptEdgeSetByte 83
AxdiInterruptEdgeSetWord 85
AxdiInterruptEdgeSetDword 87
AxdiInterruptEdgeGetBit 89
AxdiInterruptEdgeGetByte 91
AxdiInterruptEdgeGetWord 93
AxdiInterruptEdgeGetDword 95
AxdiInterruptEdgeSet 97
AxdiInterruptEdgeGet 99

Checking the setting of input level 101

AxdiLevelSetInportBit 102
AxdiLevelSetInportByte 104
AxdiLevelSetInportWord 106
AxdiLevelSetInportDword 108
AxdiLevelGetInportBit 110
AxdiLevelGetInportByte 112
AxdiLevelGetInportWord 114
AxdiLevelGetInportDword 116
AxdiLevelSetInport 118
AxdiLevelGetInport 120

Checking the setting of output level 122

AxdoLevelSetOutportBit 123
AxdoLevelSetOutportByte 125
AxdoLevelSetOutportWord 127
AxdoLevelSetOutportDword 129
AxdoLevelGetOutportBit 131
AxdoLevelGetOutportByte 133
AxdoLevelGetOutportWord 135
AxdoLevelGetOutportDword 137
AxdoLevelSetOutport 139
AxdoLevelGetOutport 141

Reading input port 143

AxdiReadInportBit 144
AxdiReadInportByte 146
AxdiReadInportWord 148
AxdiReadInportDword 150
AxdiReadInport 152

Reading and Writing output port 154

AxdoWriteOutportBit 155
AxdoWriteOutportByte 157
AxdoWriteOutportWord 159
AxdoWriteOutportDword 161
AxdoReadOutportBit 163
AxdoReadOutportByte 165
AxdoReadOutportWord 167
AxdoReadOutportDword 169
AxdoWriteOutport 171
AxdoReadOutport 173

Advanced APIs 175

AxdiIsPulseOn 176
AxdiIsPulseOff 178
AxdiIsOn 180
AxdiIsOff 183
AxdoOutPulseOn 186
AxdoOutPulseOff 188
AxdoToggleStart 190
AxdoToggleStop 192

Error Code Table Check 194

How to Use DIO Library

DIO Library Overview

Overview

A number of signal input/output are required to run a single unit of equipment. Generally, these signal are digital signals. The control module that receives and outputs the digital signal is DIO (Digital Input Output) module.

Each DIO module has the maximum of 32 contacts and all the contacts are insulated by photo coupler. If more contacts are needed, the extension of contact is possible by using a few module. There are three types of module as follow: SIO-DI32P module that has 32 input contacts, SIO-DO32P that has 32 output contacts, and SIO-DB32P module that has 16 input contacts and 16 output contacts.

For digital input/output, not only API that detects input/output state of each bit but also API that supports input/output in the unit of 8 bit, 16 bit, and 32 bit is included. Furthermore, interrrupt can be used at both rising and falling edge for each input bit. Library API is largely divided into the following way.

• Initialization : API related to initializing module

• Checking module information : API that checks all sorts of module information

• Signal input/output : API related to checking input in the unit from each bit to 32 bit and controlling output

• Interrupt: API related to setting and checking to create interrupt at rising or falling edge for input bit

In the User's Guide, SIO-DB32P module, among several digital input/ouput modules, that has 16 input bits and 16 output bits is used for description.

Products support from AXD library as follows.

Product	Explanation	Whether Interrupt Support	Note
SIO-DB32	Input/Output 16 bit	Support	Local Board
SIO-DI32	Input 32 bit	Support	Local Board
SIO-DO32	Output 32 bit	Not support	Local Board
PCI-DB64R	Input/Output 32 bit	Support	Local Board
PCI-DI64R	Input 64 bit	Support	Local Board
PCI-DO64R	Output 64 bit	Not support	Local Board
SIO-RDI32	Input 32 bit	Not support	RTEX
SIO-RDO32	Output 32 bit	Not support	RTEX
JEPMC-IO2310	Input/Output 32 bit	Not support	Mechatrolink II

Basic rules of DIO APIs

As a basic rule for all AXL APIs, AXD which falls in DIO API group returns the result of API execution. The returned value shows if the API is performed well or not for some reason.

Do as follows to check the return value of the API.

```
DWORD Code = AxdInfoIsDIOModule(&dwStatus);

if(Code == AXT_RT_SUCCESS)
  printf("It has run properly.”);
else
  printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
```

Error Code	Description
[0000] AXT_RT_SUCCESS	API function executed successfully
[1001] AXT_RT_OPEN_ERROR	Library is not open
[1002] AXT_RT_OPEN_ALREADY	Library is already open
[1053] AXT_RT_NOT_OPEN	Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION	Hardware not supported
[1101] AXT_RT_INVALID_BOARD_NO	Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS	Invalid module position
[1103] AXT_RT_INVALID_LEVEL	Invalid level
[1151] AXT_RT_ERROR_VERSION_READ	Failed to read library version
[3001] AXT_RT_DIO_OPEN_ERROR	Failed to open DIO module
[3051] AXT_RT_DIO_NOT_MODULE	No DIO module
[3052] AXT_RT_DIO_NOT_INTERRUPT	DIO Interrupt not set
[3101] AXT_RT_DIO_INVALID_MODULE_NO	Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO	Invalid DIO OFFSET number
[3103] AXT_RT_DIO_INVALID_LEVEL	Invalid DIO level
[3104] AXT_RT_DIO_INVALID_MODE	Invalid DIO mode
[3105] AXT_RT_DIO_INVALID_VALUE	Invalid setting value
[3106] AXT_RT_DIO_INVALID_USE	Can not use DIO API function

Also, APIs take inputs of necessary arguments through parameters. For output, APIs take variable pointer and store the processed result to deliver it to user. As seen below, to execute AxdExFunction, the values for inPara1 and inPara2 are required. Do as follows if the result values are coming from outPara1 and outPara2.

```
double inPara1, inPara2;
double outPara1, outPara2;
AxdExFunction(inPara1, inPara2, &outPara1, &outPara2);
```

Description of DIO API group

AXD API can be divided into groups as follows according to the characteristics of API usage.

Prefix Categpry	Description
AxdInfo	API that checks board and module information
Axdi	Category related to Digital input
Axdo	Category related to Digital output
AxdiInterrupt	Checking the setting of input interrupt
AxdiInterruptEdge	Checking the setting of interrupt creation at input interrupt rising and falling edge
AxdiLevel	Checking the setting of input signal level
AxdiRead	Reading input signal
AxdoRead	Reading output signal
AxdoWrite	Writing output signal
AxdoLevel	Checking the setting of output signal level

Initialization and Exiting

The initialization of AXL library is done by executing AxlOpen API. If AxlOpen API is executed, not only the initialization of library internally, but also are mounted base boards and the modules initialized. Therefore, user must check if any DIO module exists before using DIO library.

Initialization of Library

● *Initialization of Library*

AxlOpen : AxlOpen is an API that initializes library. When user uses interrupt, IRQ number for processing interrupt must be registered. Especially, for base board of PCI type, IRQ number is neglected for library initialization was IRQ number is automatically created.

This API returns BOOL value, not error code so it can be used as below.

```
// Initialize library.
// 7 means IRQ. In PCI, IRQ is automatically set.
if(AxlOpen(7) == AXT_RT_SUCCESS)
  printf("Library has been initialized.");
```

● *Checking library initialization*

AxlIsOpened : AxlIsOpened is an API that checks the initialization status of library. This API also returns BOOL value, not error code. If library is initialized, it returns TRUE. If library is not initialized, it returns FALSE.

```
// Check the initialization status of library.
if(AxlIsOpened())
  printf("Library has been initialized.");
else
  printf("Library has not been initialized.");
```

● *Checking existence of DIO module*

AxdInfoIsDIOModule: AxdInfoIsDIOModule is an API to check the existence of actual DIO module. It is used to check that there is DIO module before using APIs related DIO API. To check the existence of module, do as follows.

```
// Check the existence of DIO module.
DWORD dwStatus;
AxdInfoIsDIOModule(&dwStatus);
if(dwStatus == STATUS_EXIST)
  printf("DIO module exists.");
else
  printf("DIO module does not exist.");
```

● *Checking the number of input/ouput module and the number of contact*

AxdInfoGetModuleCount: AxdInfoGetModuleCount is an API that checks the number of DIO modules mounted on the entire system.

```
// Check the number of DIO module.
long    lCount;
AxdInfoGetModuleCount(&lCount);
printf("The number of DIO module is %d.", lCount);
```

AxdInfoGetInputCount: AxdInfoGetInputCount is an API that checks the number of input contact in the designated DIO module.

```
// Check the number of input contact of module 0.
long    lCount;
AxdInfoGetInputCount(0, &lCount);
printf("The number of input contact of DIO module 0 is %d.", lCount);
```

AxdInfoGetOutputCount: AxdInfoGetOutputCount is an API that checks the number of output contact in the designated DIO module.

```
// Check the number of output contact of module 0.
long    lCount;
AxdInfoGetOutputCount(0, &lCount);
printf("The number of output contact of DIO module 0 is %d.", lCount);
```

AxdInfoGetModuleNo: AxdInfoGetModuleNo is an API that checks the number of DIO module initialized.

```
// Check the number of DIO module
long lBoardNo=0, lModulePos=0, lModuleNo;
Code = AxdInfoGetModuleNo(lBoardNo, lModulePos, & lModuleNo);
printf("The number of  DIO module is %d\n", lModuleNo);
```

Exiting Library

AxlClose : AxlClose is an API that ends library. After using library, library must be shut down to return the allotted memory. Return TRUE if library is shut down after the Successful execution of API. If not, return FALSE.

```
// Exit library.
if(AxlClose())
  printf("Library has been exited.");
```

```
// Ex1_AXD_InitAndClose.cpp : Defines the entry point for the console application.
// Initialize library, check input/output information and exit.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"
```

```
void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is automatically set by PCI.
  DWORD Code = AxlOpen(7);
  if(Code == AXT_RT_SUCCESS)
  {
     printf("Library has been initialized.\n");

     // Check the existence of DIO module
     DWORD dwStatus;
     Code = AxdInfoIsDIOModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           printf("DIO module exists.\n");

           // Check the number of DIO module
           long lModuleCounts;
           Code = AxdInfoGetModuleCount(&lModuleCounts);
           if (Code == AXT_RT_SUCCESS)
              printf("The number of DIO module : %d\n",lModuleCounts);
           else
              printf("AxdInfoGetModuleCounts() : ERROR   code 0x%x\n",Code);

           // Check the number of input/output contact
           long lInputCounts;
           long lOutputCounts;
           for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
           {
              Code = AxdInfoGetInputCount(ModuleNo,&lInputCounts);
              if(Code == AXT_RT_SUCCESS)
              {
                 DWORD Code2 = AxdInfoGetOutputCount(ModuleNo,&lOutputCounts);
                 if(Code2 == AXT_RT_SUCCESS)
                 {
                    printf("Module %d : %d input contacts, %d output contacts\n",
                       ModuleNo,lInputCounts,lOutputCounts);
                 }
                 else
                    printf("AxdInfoGetOutputCounts() : ERROR   code 0x%x\n",Code);
              }
              else
                 printf("AxdInfoGetInputCounts() : ERROR   code 0x%x\n",Code);
           }
        }
        else
           printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",Code);
```

```
      }
      else
         printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if(AxlClose())
      printf("Library has been exited.\n");
   else
      printf("Library has not been exited properly.\n");
}
```

Setting Signal Level of Input/Output Port

To use input/output port, signal level for each input/output contact must be set. This is to determine whether 1 is High or Low in the program. 20 APIs using different prefixes are provided to set signal level of input/output contact as shown below.

Offset Description

Offset is the serial number of input/output contacts of DIO module. Input contacts have serial number different from the output contacts.

For example, a SIO-DB32 module has total 32 contacts, 16 for input and another 16 for output. When signal is approached at bit unit, input offset has the range of 0~15 and output offset has the range of 0~15.

If the system has each SIO-DB32, SIO-DI32, and SIO-DO32 and the allotted Module No is 0,1, and 2, respecively, the relation between offset allotted for each module and each API is as follow.

Input Offset

API / Module	SIO-DB32 (In16/Out16)	SIO-DI32 (In32)	Data Size	Note
AxdiLevelSetInport AxdiLevelGetInport	0 ~ 15	16~47	1	Offset is applied to the entire module
AxdiLevelSetInportBit AxdiLevelGetInportBit	0 ~ 15	0~31	1	Apply each module Offset
AxdiLevelSetInportByte AxdiLevelGetInportByte	0 ~ 1	0~3	8	Apply each module Offset
AxdiLevelSetInportWord AxdiLevelGetInportWord	0	0 ~ 1	16	Apply each module Offset
AxdiLevelSetInportDword AxdiLevelGetInportDword	0	0	32	Apply each module Offset

Output Offset

API / Module	SIO-DB32 (In16/Out16)	SIO-DI32 (In32)	Data Size	Note
AxdoLevelSetOutport AxdoLevelGetOutport	0 ~ 15	16~47	1	Offset is applied to the entire module
AxdoLevelSetOutportBit AxdoLevelGetOutportBit	0 ~ 15	0~31	1	Apply each module Offset
AxdoLevelSetOutportByte AxdoLevelGetOutportByte	0 ~ 1	0~3	8	Apply each module Offset
AxdoLevelSetOutportWord AxdoLevelGetOutportWord	0	0 ~ 1	16	Apply each module Offset

API Description

AxdiLevelSetInport: Set signal level of the corresponding bit in bit unit at Offset address of the entire input contact module. If PC has a few DIO modules mounted, they are numbered as the order of module index. For example, if 2 SIO-DB32 modules are mounted, 16 input contacts are numbered from 0 to 15 and 16 output contacts are numbered from 16-31 for the 32 contacts of the first module. The API that sets signal level of input contact for the corresponding bit by using this Offset is AxdiLevelSetInport.

If you want to set the signal level of bit 20 among the entire input contacts as HIGH, do as follows.

```
// Set the level of data in bit of Offset 20 of entire module as HIGH.
AxdiLevelSetInport(20, 1);
```

AxdoLevelSetOutport: This is an API that sets signal level of output contact. The method of use is the same as AxdiLevelSetInport.

AxdiLevelSetInportBit: Set the signal level of the corresponding bit in bit unit at input contact offset address of the DIO module that user designates. As described before, the offset address of input contact is the serial number for input contact. For an API that designates a specific module, the address means the serial number endowed for input contacts inside one module. For example, if 2 SIO-DB32 modules are mounted, module 0 has input offset vlaues from 0 to 15 and module 1 also has different offset values from 0 to 15.

If you want to set the signal level of bit 1 of module 0, do as follows.

```
// At Offset 0 in module 0, set data level HIGH in bit unit.
AxdiLevelSetInportBit(0, 0, 1);
```

AxdoLevelSetOutportBit: It is an API that sets signal level of output contact. The method of use is the same as AxdiLevelSetInportBit, input API.

AxdiLevelSetInportByte: At input offset address of the DIO module that user sets, set the signal level of the corresponding bit in byte unit. (8 contacts at the same time)The offset address is the port number that the corresponding input contact belongs. Offset value takes value from 0 to 3. Signal leverl can be set in byte of 8 bit. SIO-DB32 module has the effective offset range for one module as 0~1.

If you want to set the signal level at offset 0 of module 0, do as follows.

```
// At Offset 0 in module 0, set data level in byte unit as HIGH.
AxdiLevelSetInportByte(0, 0, 0xFF);
```

AxdoLevelSetOutportByte: It is an API that sets signal level of output contact. The method of use is the same as AxdiLevelSetInportByte, an input API.

AxdiLevelSetInportWord: AxdiLevelSetInportWord is similar to the previously described AxdiLevelSetInportByte. The differece is that it sets signal level of contact in word unit(16 contacts). As it uses 16 contacts as bundle, offset value is either 0 or 1. For SIO-DB32 module, the effective offset is only 0, Even in AxdSetPortLevelWord API, if you want to set the signal level of a specific offset, do as follows.

```
// At Offset 0 in module 0, set all data levels in word unit as HIGH.
AxdiLevelSetInportWord(0, 0, 0xFFFF);
```

AxdoLevelSetOutportWord: It is an API that sets signal level of output contact. The method of use is the same as AxdiLevelSetInportWord, an input API.

AxdiLevelSetInportDword: AxdiLevelSetInportDword is similar to the previously described AxdiLevelSetInportWord. The difference is that it sets signal level of contact in Dword unit (32 contacts). As it uses 32 contacts as bundle, offset value is either 0 or 1.

Even in AxdiLevelSetInportWord API, if you want to set the signal level of a specific offset, do as follows.

```
// At Offset 0 in module 0, set all data level in dword unit as HIGH.
AxdiLevelSetInportDword(0, 0, 0xFFFFFFFF);
```

AxdoLevelSetOutportDword: It is an API that sets signal level of output contact. The method of use is the same as AxdiLevelSetInportDword, an input API.

Reading Input/Output Port

API Description

To check the current state of input/output port in a specific DIO module, there are 5 input APIs and output APIs each.

AxdiReadInport: Read the corresponding bit information in bit unit at input contact offset address of the entire DIO module mounted on the system and return 1 or 0.

If you want to check bit 1 in module 0, do as follows.

```
// Read data in bit unit at Offset 1 in module 0.
DWORD dwValue;
AxdiReadInport(0, 1, &dwValue);
```

AxdoReadOutport: It is an API that reads the state of output contact. The method of use is the same as AxdiReadInport, an input API.

AxdiReadInportBit: Read bit information at input offset address of the DIO module that user designates in bit unit and return 0 or 1.

If you want to check bit value of bit 1 in module 0, do as follows.

```
// Read data at offset 1 in module 0 in bit unit.
DWORD dwValue;
AxdiReadInportBit(0, 1, &dwValue);
```

AxdoReadOutportBit: It is an API that reads the state of output contact. The method of use is the same as AxdiReadInportBit, an input API.

AxdiReadInportByte: Read data at input offset address of the module that user designates in byte unit. Offset address has the value of 0,1,2, and 3. As AxdiReadInportByte API return value in byte unit, if you want to check the signal for a certain bit, perform bit operation as below.

```
// Read data at Offset 0 in module 0 in byte unit.
DWORD dwValue;
AxdiReadInportByte(0, 0, &dwValue);

// Check the information of input bit 0 in module 0 by using bit operation.
BOOL Bit0 = dwValue&0x01 ? 1 : 0 ;
BOOL Bit1 = dwValue&0x02 ? 1 : 0 ;
BOOL Bit2 = dwValue&0x04 ? 1 : 0 ;
BOOL Bit3 = dwValue&0x08 ? 1 : 0 ;
```

AxdoReadOutportByte: It is an API that reads the state of output contact. The method of use is the same as AxdiReadInportByte, an input API.

AxdiReadInportWord: Read data at input offset address of the module that user designates in word unit. Offset address has the value of 0 and 1. As AxdiReadInportWord API returns value in word unit, if you want to check the signal for a certain bit, perform bit operation as below.

```
// Read data at Offset 0 in module 0 in word unit.
DWORD dwValue;
AxdiReadInportWord(0, 0, &dwValue);

// Check the information of input bit 1 in module 0 by using bit operation.
BOOL Bit1 = dwValue&0x0002 ? 1 : 0 ;
```

AxdoReadOutportWord: It is an API that reads the state of output contact. The method of use is the same as AxdiReadInportWord, an input API.

AxdiReadInportDword: Read data at input offset address of the module that user designates in dword unit. Offset address has the value of 0. As AxdiReadInportDword API returns value in dword unit, if you want to check the signal for a certain bit, perform bit operation as below.

```
// Read data at Offset 0 in module 0 in dword unit.
DWORD dwValue;
AxdiReadInportDword(0, 0, &dwValue);

// Check the information of input bit 2 in module 0 by using bit operation.
BOOL Bit3 = dwValue&0x00000004 ? 1 : 0 ;
```

AxdoReadOutportDword: It is an API that reads the state of output contact. The method of use is the same as AxdiReadInportDword, an input API.

```
// Ex2_AXD_ReadPort.cpp : Defines the entry point for the console application.
// Check the information on the input port of DIO module.
// Assume that module 0 as input contact module 32.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is automatically set by PCI.
  DWORD Code = AxlOpen(7);
  if(Code == AXT_RT_SUCCESS)
  {
     printf("Library has been initialized.\n");

     // Check the existence of DIO module.
     DWORD dwStatus;
     Code = AxdInfoIsDIOModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
```

```
// Check the number of DIO module.
long lModuleCounts;
AxdInfoGetModuleCount(&lModuleCounts);
// Set input/ouput signal level of DIO module as High.
for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
{
   if(AxdInfoGetInputCount(ModuleNo) > 0)
   {
      AxdiLevelSetInportDword(ModuleNo,0,0xffffffff);
   }
   if(AxdInfoGetOutputCount(ModuleNo) > 0)
   {
      AxdoLevelSetOutportDword(ModuleNo,0,0xffffffff);
   }
}

// Return the current state of Port while going round the infinite loop.
printf("INFORMATION*****************\n");
printf("Press any key to exit. \n");
printf("***************************\n");
printf("\n\n The current state of input port of module 0.\n");

BOOL fExit = FALSE;
DWORD Status_Module0[32];
while(!fExit)          // Infinite loop
{
   if(kbhit())       // Press any key to exit infinite loop.
      fExit = TRUE;

   // Check the state of the corresponding offset in bit unit.
   AxdiReadInportBit(0,0,&Status_Module0[0]);
   AxdiReadInportBit(0,1,&Status_Module0[1]);
   // Check the state of the corresponding offset in byte unit.
   AxdiReadInportByte(0,0,&Status_Module0[2]);
   AxdiReadInportByte(0,0,&Status_Module0[3]);
    // Check the state of the corresponding offset in word unit.
   AxdiReadInportWord(0,0,&Status_Module0[4]);
   AxdiReadInportWord(0,0,&Status_Module0[5]);

    // Check the state of the corresponding offset in dword unit.
   AxdiReadInportDword(0,0,&Status_Module0[16]);
   AxdiReadInportDword(0,0,&Status_Module0[17]);
   // Check the state of the corresponding offset in byte unit.
    AxdiReadInportByte(0,3,&Status_Module0[18]);
    AxdiReadInportByte(0,3,&Status_Module0[19]);
   // Check the state of the corresponding offset in word unit.
   AxdiReadInportWord(0,1,&Status_Module0[20]);

   // Return the current state of Input Port.
```

```
            printf("\r0[%d],1[%d],2[%d],3[%d],4[%d].. 18[%d],19[%d],20[%d]...",
               Status_Module0[0],Status_Module0[1], Status_Module0[2]&0x04 ? 1 : 0,
               Status_Module0[3]&0x08 ? 1 : 0, Status_Module0[4]&0x0010 ? 1 : 0,
               Status_Module0[5]&0x0020 ? 1 : 0, Status_Module0[16]&0x00010000 ? 1 : 0,
               Status_Module0[17]&0x00020000 ? 1 : 0, Status_Module0[18]&0x04 ? 1 : 0,
               Status_Module0[19]&0x08 ? 1 : 0, Status_Module0[20]&0x0010 ? 1 : 0);
         }
      }
      else
         printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
   }
   else
      printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
   printf("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if(AxlClose())
   printf("Library has been exited.\n");
else
   printf("Library has not been exited properly.\n");
}
```

Writing Output Port

Output Port is the port that outputs signal from module. SIO-DO32 has 32 output contacts. SIO-DB32 module has 16 output contacts. To output signal using Output Port, the following 5 APIs are provided.

API Description

AxdoWriteOutportBit: Send out output from the corresponding output contact at output offset address of the DIO module that user designates in bit unit.

If you want to send out output through output contact 1 in module 0, do as follows.

```
// Output data at output contact 1(Offset : 1)  in module 0 in bit unit.
AxdoWriteOutportBit(0, 1, 1);
```

AxdoWriteOutportByte: Output data from the output offset address of the module that user designates in byte unit. Offset address has the value of 0~3. If you want to output data from output Offset address 2(Output 8 contacts from contact 16) of module 0 in byte unit, do as follows.

```
// Output data at Offset address 2 in module 0 in byte unit.
AxdoWriteOutportByte(0, 2, 0xFF);
```

AxdoWriteOutportWord: Output data from the output offset address of the module that user designates in word unit. Offset address has the value of 0~1. If you want to output data from output Offset address 0 (Output 16 contacts from contact 0) of module 0 in word unit, do as follows.

```
// Read data at Offset address 1 in module 0 in word unit.
AxdWritePortWord(0, 0, 0xFFFF);
```

AxdoWriteOutportDword: Output data from the output offset address of the module that user designates in Dword unit. Offset address has the value of 0. Output 32 output contacts at the same time.

If you want to output data from output Offset address 0 of module 0 in Dword unit, do as follows.

```
// Output data at Offset address 0 in module 0 in Dword unit.
AxdWritePortDword(0, 0, 0xFFFFFFFF);
```

AxdoWriteOutport: Send out output from the offset address of the entire output contact module mounted on the system to the corresponding bit in bit unit. If you want to output data to output contact 20 among the entire output contacts in Bit unit, do as follows.

```
// Output 1 to output contact 15 among the entire output contacts.
AxdoWriteOutport(15,1);
```

```
// Ex4_AXD_WriteOutput.cpp : Defines the entry point for the console application.
// Send out output to output port of DIO module and check the current state.

#include "stdafx.h"
```

```
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set by PCI.
   DWORD Code = AxlOpen(7);
   if(Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized.\n");

      // Check the existence of DIO module.
      DWORD dwStatus;
      Code = AxdInfoIsDIOModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            // Check the number of DIO module.
            long lModuleCounts;
            AxdInfoGetModuleCount(&lModuleCounts);

            // Set input/output level of DIO module as High.
            for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
            {
               if(AxdInfoGetInputCount(ModuleNo) > 0)
               {
                  AxdiLevelSetInportDword(ModuleNo,0,0xffffffff);
               }
               if(AxdInfoGetOutputCount(ModuleNo) > 0)
               {
                  AxdoLevelSetOutportDword(ModuleNo,0,0xffffffff);
               }
            }

            // Return the current state of Output Port while going round the infinite loop.
            printf("[INFORMATION]*************** \n");
            printf("[ESC] : Exit \n");
            printf("[0 ~ 4] : Output contact 0~4 (Output contact 0~4 of module 0) On / Off
               \n");
            printf("[5 ~ 9] : Output contact 16~20 (Output contact 0~4 of module 1) On / Off
               \n");
            printf("**************************** \n");
            printf("\n\ the current state of Output Port \n");

            BOOL fExit = FALSE;
            while(!fExit)           // Infinite loop
```

```
{
   if(kbhit())       // Press any key
   {
      int ch = getch();
      DWORD OutPort;
      switch(ch)
      {
         case 27:          // Esc key
            fExit = TRUE;
         break;

         // On/Off the corresponding offset address among the entire output bit.
         case '0':
            AxdoReadOutport(0,&OutPort);
            if(OutPort) AxdoWriteOutport(0,0);
            else AxdoWriteOutport(0,1);
         break;

         case '1':
            AxdoReadOutport (1,&OutPort);
            if(OutPort) AxdoWriteOutport(1,0);
            else AxdoWriteOutport(1,1);
         break;

         case '2':
            AxdoReadOutport (2,&OutPort);
            if(OutPort) AxdoWriteOutport(2,0);
            else AxdoWriteOutport (2,1);
         break;

         case '3':
            AxdoReadOutport (3,&OutPort);
            if(OutPort) AxdoWriteOutport (3,0);
            else AxdoWriteOutport (3,1);
         break;

         case '4':
            AxdoReadOutport (4,&OutPort);
            if(OutPort) AxdoWriteOutport (4,0);
            else AxdoWriteOutport (4,1);
         break;

         case '5':
            AxdoReadOutport (16,&OutPort);
            if(OutPort) AxdoWriteOutport (16,0);
            else AxdoWriteOutport (16,1);
         break;

         case '6':
```

```
                    AxdoReadOutport (17,&OutPort);
                    if(OutPort) AxdoWriteOutport (17,0);
                    else AxdoWriteOutport (17,1);
                 break;

                 case '7':
                    AxdoReadOutport (18,&OutPort);
                    if(OutPort) AxdoWriteOutport (18,0);
                    else AxdoWriteOutport (18,1);
                 break;

                 case '8':
                    AxdoReadOutport (19,&OutPort);
                    if(OutPort) AxdoWriteOutport (19,0);
                    else AxdoWriteOutport (19,1);
                 break;

                 case '9':
                    AxdoReadOutport (20,&OutPort);
                    if(OutPort) AxdoWriteOutport (20,0);
                    else AxdoWriteOutport (20,1);
                 break;
              }
           }

           // Return the current state of Output Port.
           DWORD Status_Output[32];
           for(int OffsetNo=0;OffsetNo<32;OffsetNo++)
           {
              Status_Output[OffsetNo] = 0;
              AxdoReadOutport (OffsetNo,&Status_Output[OffsetNo]);
           }
           printf("\r0[%d],1[%d],2[%d],3[%d],..... 17[%d],18[%d],19[%d],20[%d]...",
              Status_Output[0], Status_Output[1], Status_Output[2], Status_Output[3],
              Status_Output[4], Status_Output[16], Status_Output[17], Status_Output[18],
              Status_Output[19], Status_Output[20]);
        }
     }
     else
        printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
  }
  else
     printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
  printf("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if(AxlClose())
```

```
      printf("\n Library has been exited.\n");
   else
      printf("\n Library has not been exited properly.\n");
}
```

Interrupt

AXD provides APIs to set interrupt respectively for Rising Edge(Up Edge) and Falling Edge(Down Edge) for all input contacts. To use Interrupt, the location for Interrupt to occur must be set by Interrupt Rising Edge Register와Interrupt Falling Edge Register in the first place. Then, the method to process the interrupt is set. As AXD supports call back API method, window message method, and event method as methods to process interrupt, choose one of them that fit the development environment for user. Finally, set ENABLE to use interrupt.

Setting Interrupt Rising / Falling Edge

Interrupt Rising / Falling Edge Register is to set whether or not to create Interrupt at Rising Edge or Falling Edge of the corresponding input contact. If the corresponding bit of the register is set as 1, Interrupt is created when the corresponding Edge occurrs. For example, if Interrupt Rising Edge Register of offset address 0 is set as 1, Interrupt is created when Rising Edge occurrs at input contact 0.

As Rising Edge and Falling Edge are relative concepts, if signal level is set as HIGH, Rising Edge is created as it changes from LOW with no signal to HIGH with signal. If the signal goes away again, Falling edge is created as it turns back to LOW. On the contrary, if signal level is set as LOW, Falling Edge is created when signal come in and Rising Edge is created when signal goes away.

API that sets this register or checks the setting value includes one that sets or checks bit corresponding to a specific offset address among the entire input contacts and the other that sets or checks input contact corresponding to offset address in bit, byte, word, and dword unit.



Axd: DIO API	Edge: Rising Falling Edge register setting	Set: Setting Mode Get: Checking Mode	None: Checking a specific value among the entire values bit: The unit of 1 contact byte: The unit of 8 contacts word: The unit of 16 contacts dword: The unit of 32 ocntacts

AxdiInterruptEdgeSet: AxdiInterruptEdgeSet is an API that sets register to create Interrupt when Rising or Falling Edge occurrs at input contact in a specific offset address among the entire input contacts. Offset address has the value of the total number of input contact-1. For instance, if 2 SIO-DB32 modules are installed, 16 input contacts of module 0 have offset value from 0 to 15. Another 16 input contacts of module 1 have offset value from 16 to 31. According to Edge to set, input UP_EDGE or DOWN_EDGE. If the setting value is 1, Interrupt is created. If the value set is 0, no Interrupt is created.

If you want to create Interrupt at Rising Edge of input contact 2 among the entire input contacts,do as follows.

```
// Set Interrupt at Upedge of input contact 2 among the entire input contacts.
AxdiInterruptEdgeSet(2,UP_EDGE,1);
```

AxdiInterruptEdgeSetBit: AxdiInterruptEdgeSetBit is an API that sets register to create Interrupt when Rising or Falling Edge occurrs at input contact in a specific offset address in a specific module. Offset address has the value from 0 to the total number of input contact-1. For instance, if 2 SIO-DB32 modules are installed, 16 input contacts of module 0 have offset value from 0 to 15. Another 16 input contacts of module 1 have offset value from 0 to 15. As seen in the previous description, according to Edge to set, input UP_EDGE or DOWN_EDGE. If the value set is 1, Interrupt occurs. If the setting value is 0, no Interrupt occurs.

If you want to create Interrupt at Falling Ege of input contact 2 of module 2, do as follows.

```
// Set Interrupt at Upedge of input contact 2 of module 0.
AxdiInterruptEdgeSetBit(0,2,DOWN_EDGE,1);
```

The Interrupt setting of input contact corresponding to a specific offset address in a specific module can be made in bit unit, byte unit consisting of 8 contacts, word unit consisting of 16 contacts, and dword unit cosisting of 32 contacts. As Default value is set as 0, it is acceptable to set only bits for use as 1. It is useful for the case that accurage setting is required or the nubmer of bit to be set is large.

If you want to create Interrupt only when Rising Edge occurrs at input contact of odd nubmer with SIO-DB32 module as module 0, do as follows.

```
// Set Interrupt at Upedge of input contact of odd number among the input contacts
// of module 0.
AxdiInterruptEdgeSetWord(0,0,UP_EDGE,0xAAAA);
```

Checking the Set Value for Interrupt Rising / Falling Edge

APIs that are given to check the set value at Interrupt Rising/Falliing Edge register as follows: AxdiInterruptEdgeGet API that checks the setting of Rising Edge of input contact as a specific offset address in the entire input bit, AxdiInterruptEdgeGetBit API that checks the value for a specific offset address in a specific module in bit unit, AxdiInterruptEdgeGetByte API that that checks in byte unit consisting of 8 contacts, AxdiInterruptEdgeGetWord API that checks in word unit consisting of 16 contacts, and AxdiInterruptEdgeGetDword API that checks in dword consisting of 32 contacts.

AxdiInterruptEdgeGet: If you want to check the value set for Rising Edge Register of input contact 2 among the entire input contact, do as follows. As in AxdiInterruptEdgeSet API, offset value has the range from 0 to the number of entire contact-1.

```
// Check the set value for RisingEdge Register of input contact 2 among the entire
// input contact.
DWORD dwEdge;
AxdiInterruptEdgeGet(2,UP_EDGE,&dwEdge);
```

AxdiInterruptEdgeGetBit: If you want to check the set value for Falling Edge Register of input contact 2 among input contacts of module 0, do as follows. As in AxdiInterruptEdgeSetBit API, offset value has the range from 0 to the number of contact of the corresponding module -1.

```
// Check the value set for FallingEdge Register of input contact 2 of module 0 among input
// contacts of module 0.
DWORD dwEdge;
AxdiInterruptEdgeGetBit(0,2,DOWN_EDGE,&dwEdge);
```

Offset values for AxdiInterruptEdgeGetByte, AxdiInterruptEdgeGetWord, and AxdiInterruptEdgeGetDword API are the same for the corresponding Set APIs. Thus, use the Set APIs as reference.

Setting the Use of Interrupt

Default for using Interrupt is set as DISABLE. Therefore, it must be set again as ENABLE to create Interrupt at Rising/Falling Edge. The API that determines the setting whether to use Interrupt is AxdiInterruptSetModuleEnable.

If you want to set Interrupt at module 0 as ENABLE, do as follows.

```
// Set Interrupt at module 0 as ENABLE.
AxdiInterruptSetModuleEnable(0,ENABLE);
```

Setting the Process Method for Interrupt

The methods to process Interrupt created are call back API method, window message method, and event method. Call back API method has the advantage that it can receive the event fastest as call back API is called up right at the moment event occurrs. Yet, main process experiences traffic until user exits call back API. In other words, keep the fact that traffic can happen when performing work that impose considerable load on call back API. However, event method uses a specific thread that checks whether event is created thus operates independently of main process, As it can make the best use of resource in MultiProcessor system, it is highly recommended for such use.

To set the method to process event, do as follows using AxdiInterruptSetModule API.

1) Call back API method

Set as below to use call back API method.

```
// Set the event process method for module 0 as call back method.
AxdiInterruptSetModule(0,NULL,NULL,OnDIOInterruptCallback,NULL);
```

OnDIOInterruptCallback is the pointer of the call back API that processes Interrupt. Call back API is global API and can be set as below. When Interrupt is created, call back API that will process Interrupt is automatically called up.

```
void CExDIOInterruptDlg::OnBtnStartInterruptCallback()
{
  // Setting the method to process Interrupt.
  AxdiInterruptSetModule(MODULE_NO,NULL,NULL,OnDIOInterruptCallback,NULL);

  // Setting Interrupt mask for a specific module in Dword unit.
  AxdiInterruptEdgeSetDword(MODULE_NO,0,UP_EDGE,0xFFFFFFFF);
```

```
   // Setting whether to allow Interrupt.
   AxdiInterruptSetModuleEnable(MODULE_NO,ENABLE);
}

// Call back API that will process Interrupt
void OnDIOInterruptCallback(long nModuleNo, DWORD uFlag)
{
   // nModuleNo : Module number where Interrupt is created.
   // uFlag : Flag for Interrupt created.

   DWORD dwValue;
   CString IntMessage;
   Int InputCounts = 32;     // The number of entire input contact

   for(int ChkBit=0; ChkBit<InputCounts; ChkBit++)
   {
      if((uFlag >> ChkBit) & 0x01)
      {
         AxdiReadInportBit(nModuleNo, ChkBit, &dwValue);
         if(dwValue)
            printf("INTERRUPT(MESSAGE) :\n Rising Interrupt is created at input bit %d of
               module %d.", nModuleNo,ChkBit);
         else
            printf("INTERRUPT(MESSAGE) :\n Falling Interrupt is created at input bit %d of
               module %d ." nModuleNo,ChkBit);
      }
   }
}
```

2) Window message method

To use window message method, perform setting as below. C does not support window message method. Thus, this part is described by VC++. Input window handle and window message that will receive message to use window message method. If you do not want to use window message method or do want to use default value, input 0.

```
// Set the method to process event for module 0 as window message method.
AxdiInterruptSetModule(lModuleNo,this->m_hWnd,WM_DIO_INTERRUPT,NULL,NULL);
```

The overview of using is as below. OnDIOInterruptMessage API processes Interrupt.

```
#define WM_DIO_INTERRUPT 1011   // Define message that receives Interrupt message.
#define MODULE_NO 0             // DIO module number

…(Omitted)…

// Connect message and API
BEGIN_MESSAGE_MAP(CExDIOInterruptDlg, CDialog)
```

```
   //{{AFX_MSG_MAP(CExDIOInterruptDlg)

   ON_MESSAGE(WM_DIO_INTERRUPT,OnDIOInterruptMessage)

   //}}AFX_MSG_MAP
END_MESSAGE_MAP()

…(Omitted)…

void CExDIOInterruptDlg::OnBtnStartInterruptMessage()
{
   // Set the method to process Interrupt (Window message method).
   AxdiInterruptSetModule(MODULE_NO,this->m_hWnd,WM_DIO_INTERRUPT,NULL,NULL);

   // Set Interrupt mask for module 0 in Dword unit.
   AxdiInterruptEdgeSetDword(MODULE_NO,0,UP_EDGE,0xFFFFFFFF);

   // Set whether to allow Interrupt.
   AxdiInterruptSetModuleEnable(MODULE_NO,ENABLE);
}

void CExDIOInterruptDlg::OnDIOInterruptMessage(WPARAM nModule, LPARAM nIntrFlag)
{
// nModule : Module number where Interrupt is created ( 0, 1, 2, 3)
// nIntrFlag : Flag where Interrupt is created (Input Port checks that Interrupt is created)

   DWORD dwValue;
   CString IntMessage;
   int InputCounts = 16; // The total number of input Bit

   for(int ChkBit=0; ChkBit<InputCounts; ChkBit++)
   {
      if((nIntrFlag >> ChkBit) & 0x01)
      {
         AxdiReadInportBit(nModule, ChkBit, &dwValue);
         if(dwValue)
            IntMessage.Format("INTERRUPT(MESSAGE) : Rising Interrupt is created at input
               bit %d of module %d",nModule,ChkBit);
         else
            IntMessage.Format("INTERRUPT(MESSAGE) : Falling Interrupt is created at input
               bit %d of module %d",nModule,ChkBit);

         m_LIST_Message.InsertItem(0,IntMessage);
      }
   }
}
```

3) Event method

To use event method, perform the setting as below. Event method uses a specific thread that checks whether event is created thus operates independently of main process, As it can make the best use of resource in MultiProcessor system, it is highly recommended for such use.

This part is also described by using VC++. To use event method, m_hInterruptEvent, HANDLE type member variable, must be declared ahead first of all.

```
// Set the method to process event for module 0 as event method.
AxdiInterruptSetModule(0,NULL,NULL,NULL,&m_hInterruptEvent);
```

The overview of using is as below. ThreadProc API processes Interrupt. Here, keep the fact that AxdiInterruptSetModule API must be called up before thread is executed in mind.

When Interrupt is created, Interrupt event occurrs. Thread detects the event to recognize the creation of Interrupt. AxdReadInterrupt is called up inside thread to check the module number where Interrupt is created and Interrupt flag information. After this, it checks in which module, and which input bit the Interrupt is created.

```
// Variable needed for Interrupt subject to event method.
HANDLE m_hThreadHandle;
BOOL m_bThread;
HANDLE m_hInterruptEvent;
```

```
void CExDIOInterruptDlg::OnBtnStartInterruptEvent()
{
  // Set the method to process Interrupt.
  AxdiInterruptSetModule(0,NULL,NULL,NULL,&m_hInterruptEvent);  // Event method.

  // Set Interrupt mask for a specific module in Dword unit.
  AxdiInterruptEdgeSetDword(MODULE_NO,0,UP_EDGE,0xFFFFFFFF);

  // Set whether to allow Interrupt.
  AxdiInterruptSetModuleEnable(MODULE_NO,ENABLE);

  // Create thread to use event method.
  m_bThread = TRUE;
  CreateThread(NULL, 0, (LPTHREAD_START_ROUTINE)ThreadProc, this, NULL, NULL);
}

void ThreadProc(LPVOID lpData)
{
  CExDIOInterruptDlg *pDlg  = (CExDIOInterruptDlg *)lpData;
  long  lModuleNo;
  DWORD dwFlag;
  DWORD dwValue;
  CString IntMessage;
  int InputCounts = 32;              // The total number of input Bit.

  while(pDlg->m_bThread)
```

```
    {
        if(WaitForSingleObject(pDlg->m_hInterruptEvent, 0) == WAIT_OBJECT_0)
        {
            // Read Interrupt information if it is event method.
            AxdiInterruptRead(&lModuleNo, &dwFlag);
            for(int ChkBit=0; ChkBit<InputCounts; ChkBit++)
            {
                if((dwFlag >> ChkBit) & 0x01)
                {
                    AxdiReadInportBit(lModuleNo, ChkBit, &dwValue);
                    if(dwValue)
                        IntMessage.Format("INTERRUPT(MESSAGE) : Interrupt is created at input
                            bit %d of module %d",lModuleNo,ChkBit);
                    else
                        IntMessage.Format("INTERRUPT(MESSAGE) : Falling Interrupt is created at
                            input bit %d of module %d",lModuleNo,ChkBit);
                    pDlg->m_LIST_Message.InsertItem(0,IntMessage);
                }
            }
        }
    }
}
```

```
// Ex5_AXD_InterruptUsing.cpp : Defines the entry point for the console application.
// Create Interrupt and check current state.

#include "stdafx.h"
#include "stdio.h"
#include <conio.h>
#include "AXL.h"

// Callback API to handle Interrupt
void OnDIOInterruptCallback(long nModuleNo, DWORD uFlag)
{
    // nModuleNo    : number of interrupt occurred module, uFlag    : interrupt occurred Flag

    DWORD dwValue;
    int InputCounts = 32;     // number of entire input contact

    for(int ChkBit=0; ChkBit<InputCounts; ChkBit++)
    {
        if((uFlag >> ChkBit) & 0x01)
        {
            AxdiReadInportBit(nModuleNo, ChkBit, &dwValue);
            if (dwValue)
                printf("\n Rising Interrupt is created at input bit %d of module %d.",
                    nModuleNo,ChkBit);
            else
```

```
            printf("\n Falling Interrupt is created at input bit %d of module %d.",
               nModuleNo,ChkBit);
      }
   }
}

void main()
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set by PCI.
   DWORD Code = AxlOpen(7);
   if(Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized.\n");

      // Check the existence of DIO module.
      DWORD dwStatus;
      Code = AxdInfoIsDIOModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
         {
            // Check the number of DIO module.
            long lModuleCounts;
            int ModuleNo;
            AxdInfoGetModuleCount(&lModuleCounts);

            // Set input/ouput level of DIO module as High.
            for(ModuleNo=0; ModuleNo < lModuleCounts; ModuleNo++)
               AxdSetPortLevelDword(ModuleNo,0,0xffffffff);

            printf("[INFORMATION]*************** \n");
            printf("[ESC] : Exit \n");
            printf("*************************** \n\n");

            // Set Interrupt.
            for(ModuleNo=0; ModuleNo < lModuleCounts; ModuleNo++)
            {
               // Set the method to process Interrupt. (Call back method)
               AxdiInterruptSetModule(ModuleNo,NULL,NULL, OnDIOInterruptCallback, NULL);
               // Set Interrupt mask of a specific module in Dword unit
               // (1 for both Rising/Falling).
               AxdiInterruptEdgeSetDword(ModuleNo,0,UP_EDGE,0xFFFFFFFF);
               // Set whether to allow Interrupt.
               AxdiInterruptSetModuleEnable(ModuleNo,ENABLE);
            }

            BOOL fExit = FALSE;
            while(!fExit)     // Infinite loop
```

```
            {
               if(kbhit())    // Press any key
               {
                  int ch = getch();
                  switch(ch)
                  {
                     case 27:    // Esc key
                        fExit = TRUE;
                     break;
                  }
               }
            }
         }
         else
            printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",Code);
      }
      else
         printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
   }
   else
      printf("AxlOpen() : ERROR  code 0x%x\n",Code);

   // Exit library.
   if(AxlClose())
      printf("\nLibrary has been exited.\n");
   else
      printf("\nLibrary has not been exited properly.\n");
}
```

Advanced DIO API

AXL library provides APIs not only for the abovementioned basic input/output, but also APIs that perform several advanced DIO function which is frequently used on site.

Checking Signal Variance

AxdiIsPulseOn: It is an API that checks whether signal coming from input contact which is a specific input offset number of the desiganted module is turned on from off. To check the variance of signal, AxdiIsPulseOn API must be called up at least 2 times. It returns TRUE if signal level when it was called up is Off and is On now. If not, it returns FALSE.

If you want to check that signal of input contact which is offset 0 of module 0 is turned On from Off, do as follows.

```
// Check if signal of input contact, which is offset 0 of module 0, is turned On from Off.
DWORD uValue;
AxdiIsPulseOn(0,0,&uValue);
if(uValue)
  printf("Signal of input contact, which is offset 0 of module 0, is turned On from Off.");
```

AxdiIsPulseOff: It is an API that checks whether a signal coming from input contact, which is a specific input offset number of the designated module, is turned Off from On. The API returns TRUE if signal level that was called up before was On and is Off now. If not, it returns FALSE. If you want to check if signal of input contact which is offset 1 of module 0 is turned Off from On, do as follows.

```
// Check if signal of input contact, which is offset 1 of module 0, is turned Off from On.
DWORD uValue;
AxdiIsPulseOff(0,1,&uValue);
if(uValue)
  printf("Check if signal of input contact, which is offset 1 of module 0, is turned Off
     from On.");
```

```
// Ex6_AXD_IsPulseOnOff.cpp : Defines the entry point for the console application.
// Check signal variance at a specific input port.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
  // Initialize library.
  // 7 means IRQ.  IRQ is automatically set by PCI.
  DWORD Code = AxlOpen(7);
```

```
if(Code == AXT_RT_SUCCESS)
{
   printf("Library has been initialized.\n");
   // Check the existence of DIO module.
   DWORD dwStatus;
   Code = AxdInfoIsDIOModule(&dwStatus);
   if(Code == AXT_RT_SUCCESS)
   {
      if(dwStatus == STATUS_EXIST)
      {
         // Check the number of DIO module.
         long lModuleCount;
         AxdInfoGetModuleCount(&lModuleCount);

         // Set input/output signal level of DIO module as High.
         for(int ModuleNo=0;ModuleNo<lModuleCount;ModuleNo++)
            AxdSetPortLevelDword(ModuleNo,0,0xffffffff);

         // Return the current status of Output Port while going round the infinite loop.
         printf("[INFORMATION]*************** \n");
         printf("[ESC] : Exit \n");
         printf("*************************** \n");

         BOOL fExit = FALSE;
         while(!fExit)     // Infinite loop
         {
            if(kbhit()) // Press any key
            {
               int ch = getch();
               switch(ch)
               {
                  case 27:    // Esc key
                     fExit = TRUE;
                  break;
               }
            }

            // Return the current state of Input Port.
            DWORD Status_Input;
            DWORD uValue;
            AxdiReadInport(0,&Status_Input);
            printf("\rThe current state of offset 0 of module 0 : %d",Status_Input);

            // Check if signal of input contact, which is offset 0 of module 0,
            // is turned On from Off.

            AxdiIsPulseOn(0,0,&uValue);
            if(uValue)
               printf("\n Signal of input contact, which is offset 0 of module 0, is
```

```
                    turned On from Off.\n");

             // Check if signal of input contact, which is offset 0 of module 0,
             // is turned Off from On.

             AxdiIsPulseOff(0,0,&uValue);
             if(uValue)
                printf("\n Signal of input contact, which is offset 0 of module 0, is
                   turned Off from On.\n");
          }
       }
       else
          printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
    }
    else
       printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE ) code 0x%x\n",Code);
  }
  else
    printf("AxlOpen() : ERROR code 0x%x\n",Code);

  // Exit library.
  if(AxlClose())
    printf("\nLibrary has been exited.\n");
  else
    printf("\n Library has not been exited properly.\n");
}
```

Checking Signal Continuity

AxdiIsOn: It is an API that checks whether signal coming from input contact, which is a specific offset number of the designated module, keeps on while AxdiIsOn API is called up n times. It returns TRUE at the last nth call-up, if signal of the input contact is On during the moment that AxdiIsOn API is called up n times. If not, it returns FALSE. The return value until n-1th call-up is FALSE.

If you want to check whether signal of input contact, which is offset 0 of module 0, keeps On during the time when the API is called up 10 times, do as follows.

```
// Check whether signal of input contact, which is offset 0 of module 0, keeps On during
// the time when the API is called up 10 times.
DWORD uValue;

AxdiIsOn(0,0,10,&uValue, 1);          // Initial start
for(int i = 0; i < 9; i++)
{
  // In the actual program, call-up routine that has constant period through Timer or
  // Thread is used.
  AxdiIsOn(0,0,10,&uValue, 0);
}
```

```
if(uValue)
   printf("Signal of input contact, which is offset 0 of module 0, has kept On.");
```

AxdiIsOff: It is an API that checks whether signal coming from input contact, which is a specific offset number of the designated module, keeps Off while AxdiIsOff API is called up n times. It returns TRUE at the last nth call-up, if signal of the input contact is Off during the moment that AxdiIsOff API is called up n times. If not, it returns FALSE. The return value until n-1th call-up is FALSE.

If you want to check whether signal of input contact, which is offset 0 of module 0, keeps Off during the time when the API is called up 10 times, do as follows.

```
// Check whether signal of input contact, which is offset 0 of module 0, keeps Off during
// the time when the API is called up 10 times.
DWORD uValue;

AxdiIsOff(0,0,10,&uValue, 1);              // Initial start
for(int i = 0; i < 9; i++)
{
   // In the actual program, call-up routine that has constant period through Timer or
   // Thread is used.
   AxdiIsOff(0,0,10,&uValue, 0);
}
if(uValue)
   printf("Signal of input contact, which is offset 0 of module 0, has kept Off.");
```

```
// Ex7_AXD_IsOnOff.cpp : Defines the entry point for the console application.
// Check the signal continuity at a specific input contact.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set by PCI.
   DWORD Code = AxlOpen(7);
   if(Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized. \n");

      // Check the existence of DIO module.
      DWORD dwStatus;
      Code = AxdInfoIsDIOModule(&dwStatus);
      if(Code == AXT_RT_SUCCESS)
      {
         if(dwStatus == STATUS_EXIST)
```

```
{
   // Check the number of DIO module.
   long lModuleCount;
   AxdInfoGetModuleCount(&lModuleCount);

   // Set input/output signal level of DIO module as High.
   for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
   {
      if(AxdInfoGetInputCount(ModuleNo) > 0)
      {
         AxdiLevelSetInportDword(ModuleNo,0,0xffffffff);
      }
      if(AxdInfoGetOutputCount(ModuleNo) > 0)
      {
         AxdoLevelSetOutportDword(ModuleNo,0,0xffffffff);
      }
   }

   // Return the current state of Output Port while going round the infinite loop.
   printf("[INFORMATION]*************** \n");
   printf("[ESC] : Exit \n");
   printf("[1] : Check if offset input contact 0 of module 0 keeps On during the
      moment the API is called up 100times.\n");
   printf("[2] : Check if offset input contact 0 of module 0 keeps Off during the
      moment the API is called up 100times.\n");
   printf("*************************** \n");

   BOOL fExit = FALSE;
   while(!fExit)     // Infinite loop
   {
      if(kbhit()) // Press any key
      {
         int ch = getch();
         DWORD OutPort;
         DWORD uValue;
         int nCount;
         switch(ch)
         {
            case 27:    // Esc key
               fExit = TRUE;
            break;

            case '1':
             // Check if signal of offset input contact 0 of module 0 keeps
            // On during the moment the API is called up 100times.

               AxdiIsOn(0,0,100,&uValue, 1); // Initial start
               for(nCount = 0; nCount < 99; nCount++)
               {
```

```
                        AxdiIsOn(0, 0, 100, &uValue, 0);
                    }

                    if(uValue)
                        printf("\n Signal of offset input contact 0 of module 0 has kept
                            On.\n");
                    else
                        printf("\n Signal of offset input contact 0 of module 0 has not
                            kept On.\n");
                break;

                case '2':
                // Check if signal of offset input contact 0 of module 0 keeps
                // Off during the moment the API is called up 100times.

                    AxdiIsOff(0, 0, 100, &uValue, 1);           // Initial start
                    for(nCount = 0; nCount < 99; nCount++)
                    {
                        AxdiIsOff(0, 0, 100, &uValue, 0);
                    }

                    if(uValue)
                        printf("\n Signal of offset input contact 0 of module 0 has kept
                            Off.\n");
                    else
                        printf("\n Signal of offset input contact 0 of module 0 has not
                            kept Off.\n");
                break;
            }
        }

        // Return the current state of Input Port.
        DWORD Status_Input;
        DWORD uValue;
        AxdiReadInport(0,&Status_Input);
        printf("\rThe current state of offset 0 of module 0. : %d",Status_Input);
      }
    }
    else
      printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST )  code 0x%x\n",Code);
  }
  else
    printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
}
else
  printf("AxlOpen() : ERROR  code 0x%x\n",Code);

// Exit library.
if(AxlClose())
```

```
      printf("\nLibrary has been exited.\n");
   else
      printf("\n Library has not been exited properly.\n");
}
```

Keeping Signal for a Certain Period of Time

AxdoOutPulseOn: It is an API that keeps output contact which is a specific offset number of the designated module for a certain period of time and turns the output contact off. Offset value does not start from output contact but user checks the location of output contact among the entire bit. For example, SIO-DB32 module has input contacts from 0 to 15, output contacts from 16 to 31. Thus, offset value has to use values from 16 to 31.

If you want to keep signal of output contact(offset 16) of module 0 On state for 1 second and turn if off, do as follows.

```
// Keep signal of output contact, offset 16, of module 0 On state for 1 second and
// turn if off.
AxdoOutPulseOn(0,16,1000);
```

AxdoOutPulseOff: It is an API that keeps output contact which is a specific offset number of the designated module for a certain period of time in mSec unit and turns the output contact off. As seen above, user has to check the location of output contact among the entire bit to use.

If you want to turn signal of output contact(offset 16) of module 0 On after 1 second, do as follows.

```
// Turn signal of output contact which is offset 16 of module 0 On after 1 second.
AxdoOutPulseOff(0,16,1000);
```

```
// Ex8_AXD_OutPulseOnOff.cpp : Defines the entry point for the console application.
// Keep signal on for a certain period of time at a specific output port or turn signal on
// in a certain period of time.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
   // Initialize library.
   // 7 means IRQ. IRQ is automatically set by PCI.
   DWORD Code = AxlOpen(7);
   if(Code == AXT_RT_SUCCESS)
   {
      printf("Library has been initialized.\n");

      // Check the existence of DIO module.
```

```
DWORD dwStatus;
Code = AxdInfoIsDIOModule(&dwStatus);
if(Code == AXT_RT_SUCCESS)
{
   if(dwStatus == STATUS_EXIST)
   {
      // Check the number of DIO module.
      long lModuleCount;
      AxdInfoGetModuleCount(&lModuleCount);

      // Set input/output signal level of DIO module as High.
      for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
      {
         if(AxdInfoGetInputCount(ModuleNo) > 0)
         {
            AxdiLevelSetInportDword(ModuleNo,0,0xffffffff);
         }
         if(AxdInfoGetOutputCount(ModuleNo) > 0)
         {
            AxdoLevelSetOutportDword(ModuleNo,0,0xffffffff);
         }
      }

      // Return the current state of Output Port while going round the infinite loop.
      printf("[INFORMATION]*************** \n");
      printf("[ESC] : Exit \n");
      printf("[1] : Turn signal off after a certain period of time.\n");
      printf("[2] : Turn signal off after a certain period of time.\n");
      printf("*************************** \n");

      BOOL fExit = FALSE;
      while(!fExit)     // Infinite loop
      {
         if(kbhit()) // Press any key
         {
            int ch = getch();
            long nDelay = 1000;  // [ms unit ]

            int lOffset;
            switch(ch)
            {
               case 27:    // Esc key
                  fExit = TRUE;
               break;

               case '1':
               // Turn output contact signal of module 0 off one by one.
                  for(lOffset=0;lOffset<32;lOffset++)
                     AxdoOutPulseOn(0,lOffset,nDelay*(lOffset-16));
```

```
                    break;

                    case '2':
                    // Turn output contact signal of module 0 one by one.
                       for(lOffset=0;lOffset<32;lOffset++)
                          AxdoOutPulseOff(0,lOffset,nDelay*(lOffset-16));
                    break;
                 }
              }

              // Return the current state of Output Port.
              DWORD Status_Output[32];
              for(int OffsetNo=0; OffsetNo < 32; OffsetNo++)
                 AxdoReadOutport(OffsetNo,&Status_Output[OffsetNo]);

              printf("\r16[%d],17[%d],18[%d],19[%d],20[%d],21[%d],22[%d]...",
                 Status_Output[0], Status_Output[1], Status_Output[2], Status_Output[3],
                 Status_Output[4], Status_Output[5], Status_Output[6], Status_Output[7],
                 Status_Output[8]);
           }
        }
        else
           printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
     }
     else
        printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
     printf("AxlOpen() : ERROR  code 0x%x\n",Code);

  // Exit library.
  if(AxlClose())
     printf("\nLibrary has been exited properly.\n");
  else
     printf("\n Library has not been exited properly.\n");
}
```

Signal Toggle

AxdoToggleStart: It is an API that toggles signal at output contact which is a specific ouput offset number of the designated module with the set number and interval(mSec). And it keeps the initial output state.

If you want to toggle signal of output contact 0 of module 0 10 times with the set On/Off interval time and turn the signal back to initial state, do as follows. If you want to repeat infinitely, input -1 for the repetiiton number.

```
// Toggle signal of output contact which is offset 16 of module 0 10 times with 100+200mSec
// interval. (On time 100mSec , Off Time 200 mSec)
AxdoToggleStart(0, 0, 1, 100, 200, 10);
```

AxdoToggleStop: it is an API that stops singal toggled at output contact which is a specific output offset number of the designated module and fixes it as a specific signal. Offset values are as in above.

If you want to stop the toggled signal of output contact 0 of module 0 and keep signal state On , do as follows.

```
// Stop the toggled signal of output contact 0 of module 0 and keep signal state On.
AxdoToggleStop(0, 0, 1);
```

```
// Ex9_AXD_ToggleStartStop.cpp : Defines the entry point for the console application.
// Toggle signal at a specific output port.

#include "stdafx.h"
#include "AXL.h"
#include <conio.h>
#include "stdio.h"

void main(void)
{
  // Initialize library.
  // 7 means IRQ. IRQ is automatically set by PCI.
  DWORD Code = AxlOpen(7);
  if(Code == AXT_RT_SUCCESS)
  {
     printf("Library has been initialized.\n");

     // Check the existence of DIO module.
     DWORD dwStatus;
     Code = AxdInfoIsDIOModule(&dwStatus);
     if(Code == AXT_RT_SUCCESS)
     {
        if(dwStatus == STATUS_EXIST)
        {
           // Check the number of DIO module.
           long lModuleCount;
           AxdInfoGetModuleCount(&lModuleCount);

           // Set input/output signal level of DIO module as High.
           for(int ModuleNo=0;ModuleNo<lModuleCounts;ModuleNo++)
           {
              if(AxdInfoGetInputCount(ModuleNo) > 0)
              {
                 AxdiLevelSetInportDword(ModuleNo,0,0xffffffff);
              }
              if(AxdInfoGetOutputCount(ModuleNo) > 0)
```

```
    {
        AxdoLevelSetOutportDword(ModuleNo,0,0xffffffff);
    }
}

// Return the current state of Output Port while going round the infinite loop.
printf("[INFORMATION]************** \n");
printf("[ESC] : Exit \n");
printf("[1] : Toggle signal 10 times at the interval of 1 second. \n");
printf("[2] : Stop the toggled signal and fix it as Off. \n");
printf("************************** \n");

BOOL fExit = FALSE;
while(!fExit)  // Infinite loop
{
    if(kbhit()) // Press any key
    {
        int ch = getch();
        long nDelay = 1000;  //[ms unit ]
        int lOffset;
        switch(ch)
        {
            case 27:    // Esc key
                fExit = TRUE;
            break;

            case '1':
            // Toggle output contact signal of module 0 10 times at the interval
            // of 1 second.
                for(lOffset=0;lOffset<32;lOffset++)
                    AxdoToggleStart(0,lOffset,nDelay,10);
            break;

            case '2':
            // Stop the toggled signal and keep it as Off.
                for(lOffset=0;lOffset<32;lOffset++)
                    AxdToggleStop(0,lOffset,0);
            break;
        }
    }

    // Return the current state of Output Port.
    DWORD Status_Output[32];
    for(int OffsetNo=0;OffsetNo<32;OffsetNo++)
        AxdoReadOutport(OffsetNo,&Status_Output[OffsetNo]);

    printf("\r16[%d],17[%d],18[%d],19[%d],20[%d],21[%d],22[%d]...",
        Status_Output[0], Status_Output[1], Status_Output[2], Status_Output[3],
        Status_Output[4], Status_Output[5], Status_Output[6], Status_Output[7],
```

```
                Status_Output[8]);
            }
        }
        else
            printf("AxdInfoIsDIOModule() : ERROR ( NOT STATUS_EXIST ) code 0x%x\n",Code);
    }
    else
        printf("AxdInfoIsDIOModule() : ERROR ( Return FALSE )  code 0x%x\n",Code);
  }
  else
    printf("AxlOpen() : ERROR  code 0x%x\n",Code);

  // Exit library.
  if(AxlClose())
    printf("\nLibrary has been exited properly.\n");
  else
    printf("\n Library has not been exited properly.\n");
}
```

DIO Command Manual Information

This manual is to run SIO-DI32/SIO-DO32P/SIO-DB32P/SIO-DO32T/SIO-DB32T in Windows 98, Windows NT, Windows 2000 or Windows XP OS system by Microsoft VC++6.0, Visual Basic, Borland C-Builder, and Delphi. Library APIs included are categorized by functions and described.

Header File

C++

AXD.h

Visual Basic

AXD.bas

Delphi

AXD.pas

Terms for API

API Names in the Manual

In this manual, API names used can be distinguished from each other by Prefix and Suffix for their operations.

Prefix/Suffix for API library

Prefix/Suffix	Explanation
AxdInfo	API that checks board and module information
Axdi	Category related to Digital Input
Axdo	Category related to Digital output
AxdiInterrupt	Checking the setting of input interrupt
AxdiInterruptEdge	Checking the setting of interrupt creation at input interrupt rising.falling edge
AxdiLevel	Checking the setting of input signal level
AxdiRead	Reading input signal
AxdoRead	Reading output signal
AxdoWrite	Writing output signal
AxdoLevel	Checking the setting of output signal level
Get	Checking register value
Set	Setting register value
Inport	API related to input contact
Outport	API related to output contact
Bit	Read or write contact in 1 unit
Byte	Read or write contact in 8 unit
Word	Read or write contact in 16 unit
Dword	Read or write contact in 32 unit

Argument Names in the Manual

The common arguments of the APIs used in the manual have the following meanings.

long lBoardNo: Automatic arrangement of board number in ascending order from the first board of the initialized Base Board takes place. Board number starts from '0'.

long lModuleNo: Automatic arrangement of module number in ascending order from the first module during DIO module initialization. Module number starts from '0'.

DWORD uLevel: uLevel sets and checks Level when using APIs related to ReadPort and OWriteOutport.

DWORD uOffset: uOffset has different effective range according to used module and used API. The table below can be used to set it.

Data Type	Offset	Value or Return	Related
All	0 ~ n-1	0(Off), 1(On)	
Bit	0 ~ 31	0(Off), 1(On)	API group of Bit class
Byte	0, 1, 2, 3	00h ~ FFh	API group of Byte class
Word	0, 1	0000h ~ FFFFh	API group of Word class
Dword	0	00000000h ~ FFFFFFFFh	API group of Dword class

If the system has each SIO-DB32, SIO-DI32, and SIO-DO32 module and the allotted Module No is 0, 1, and 2 for each module in the order written, the allotted Offset range for each module and the relation of each API are as follows.

Input Offset

API / Module	SIO-DB32 (In32/Out32)	SIO-DI32 (In32)	Data Size	Note
AxdiLevelSetInport AxdiLevelGetInport AxdiReadInport	0 ~ 15	16 ~ 47	1	Offset is applied to the entire module
AxdiLevelSetInportBit AxdiLevelGetInportBit AxdiReadInportBit	0 ~ 15	0 ~ 31	1	Apply each module Offset
AxdiLevelSetInportByte AxdiLevelGetInportByte AxdiReadInportByte	0 ~ 1	0 ~ 3	8	Apply each module Offset
AxdiLevelSetInportWord AxdiLevelGetInportWord AxdiReadInportWord	0	0 ~ 1	16	Apply each module Offset
AxdiLevelSetInportDword AxdiLevelGetInportDword AxdiReadInportDword	0	0	32	Apply each module Offset

Output Offset

API / Module	SIO-DB32 (In32/Out32)	SIO-DI32 (In32)	Data Size	Note
AxdoLevelSetOutport AxdoLevelGetOutport AxdoReadOutport AxdoWriteOutport	0 ~ 15	16~47	1	Offset is applied to the entire module
AxdoLevelSetOutportBit AxdoLevelGetOutportBit AxdoReadOutportBit AxdoWriteOutportBit	0 ~ 15	0~31	1	Apply each module Offset
AxdoLevelSetOutportByte AxdoLevelGetOutportByte AxdoReadOutportByte AxdoWriteOutportByte	0 ~ 1	0~3	8	Apply each module Offset
AxdoLevelSetOutportWord AxdoLevelGetOutportWord AxdoReadOutportWord AxdoWriteOutportWord	0	0 ~ 1	16	Apply each module Offset
AxdoLevelSetOutportDword AxdoLevelGetOutportDword AxdoReadOutportDword AxdoWriteOutportDword	0	0	32	Apply each module Offset

Interrupt Rising / Falling Edge Register

Interrupt Rising(Up) Edge is a register set to detect Interrupt at Rising Edge of the input bit that user wants .

If the bit is set as 1, Interrupt is created at Rising moment.

Interrupt Falling(Down) Edge is a register set to detect Interrupt at Falling Edge of the input bit that user wants .

If the bit is set as 1, Interrupt is created at Falling moment..

But, it only works only when Interrupt is set as Enable.

DIO Command Quick List

Board and Module

Function	Description
AxdInfoIsDIOModule	Check the existence of DIO module.
AxdInfoGetModuleNo	Check the number of AIO module.
AxdInfoGetModuleCount	Check the number of DIO input/output module.
AxdInfoGetInputCount	Check the number of input contact of the designated module.
AxdInfoGetOutputCount	Check the number of output contact of the designated module.
AxdInfoGetModule	Check base board number, module location and module ID by the designated module number.

Interrupt

Checking the setting of Interrupt

Function	Description
AxdiInterruptSetModule	Use window message, call back API, and event method to bring Interrupt message to the designated module.
AxdiInterruptSetModuleEnable	Set the use of Interrupt of the designated module.
AxdiInterruptGetModuleEnable	Check the use of Interrupt of the designated module.
AxdiInterruptRead	Check the location where Interrupt is created.

Checking the setting of Interrupt rising/falling edge

Function	Description
AxdiInterruptEdgeSetBit	Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSetByte	Set Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSetWord	Set Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSetDword	Set Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetBit	Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetByte	Check Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetWord	Check Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

AxdiInterruptEdgeGetDword	Check Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSet	Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.
AxdiInterruptEdgeGet	Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.

Checking the Setting of Input Level

Function	Description
AxdiLevelSetInportBit	Set data level in bit unit at Offset location of the designated input contact module.
AxdiLevelSetInportByte	Set data level in byte unit at Offset location of the designated input contact module.
AxdiLevelSetInportWord	Set data level in word unit at Offset location of the designated input contact module.
AxdiLevelSetInportDword	Set data level in double word unit at Offset location of the designated input contact module.
AxdiLevelGetInportBit	Check data level in bit unit at Offset location of the designated input contact module.
AxdiLevelGetInportByte	Check data level in byte unit at Offset location of the designated input contact module.
AxdiLevelGetInportWord	Check data level in word unit at Offset location of the designated input contact module.
AxdiLevelGetInportDword	Check data level in double word unit at Offset location of the designated input contact module.
AxdiLevelSetInport	Set data level in bit unit at Offset locations of the entire input contact module.
AxdiLevelGetInport	Check data level in bit unit at Offset locations of the entire input contact module.

Checking the Setting of Output Level

Function	Description
AxdoLevelSetOutportBit	Set data level in bit unit at Offset location of the designated output contact module.
AxdoLevelSetOutportByte	Set data level in byte unit at Offset location of the designated output contact module.
AxdoLevelSetOutportWord	Set data level in word unit at Offset location of the designated output contact module.
AxdoLevelSetOutportDword	Set data level in double word unit at Offset location of the designated output contact module.

AxdoLevelGetOutportBit	Check data level in bit unit at Offset location of the designated output contact module.
AxdoLevelGetOutportByte	Check data level in byte unit at Offset location of the designated output contact module.
AxdoLevelGetOutportWord	Check data level in word unit at Offset location of the designated output contact module.
AxdoLevelGetOutportDword	Check data level in double word unit at Offset location of the designated output contact module.
AxdoLevelSetOutport	Set data level in bit unit at Offset locations of the entire output contact module.
AxdoLevelGetOutport	Check data level in bit unit at Offset locations of the entire output contact module.

Reading Input Port

Function	Description
AxdiReadInportBit	Read data in bit unit at Offset location of the designated input contact module.
AxdiReadInportByte	Read data in byte unit at Offset location of the designated input contact module.
AxdiReadInportWord	Read data in word unit at Offset location of the designated input contact module.
AxdiReadInportDword	Read data in double word unit at Offset location of the designated input contact module.
AxdiReadInport	Read data in bit unit at Offset locations of the entire input contact module.

Reading and Writing Output Port

Function	Description
AxdoWriteOutportBit	Output data in bit unit at Offset location of the designated output contact module.
AxdoWriteOutportByte	Output data in byte unit at Offset location of the designated output contact module.
AxdoWriteOutportWord	Output data in word unit at Offset location of the designated output contact module.
AxdoWriteOutportDword	Output data in double word unit at Offset location of the designated output contact module.
AxdoReadOutportBit	Read data in bit unit at Offset location of the designated output contact module.
AxdoReadOutportByte	Read data in byte unit at Offset location of the designated output contact module.
AxdoReadOutportWord	Read data in word unit at Offset location of the designated output contact module.

AxdoReadOutportDword	Read data in double word unit at Offset location of the designated output contact module.
AxdoWriteOutport	Output data in bit unit at Offset locations of the entire output contact module.
AxdoReadOutport	Read data in bit unit at Offset locations of the entire output contact module.

Advanced API

Function	Description
AxdiIsPulseOn	Check if singal at Offset location of the designated input contact module is turned On from Off.
AxdiIsPulseOff	Check if singal at Offset location of the designated input contact module is turned Off from On.
AxdiIsOn	Check if singal at Offset location of the designated input contact module keeps On until it is called up to count number.
AxdiIsOff	Check if singal at Offset location of the designated input contact module keeps Off until it is called up to count number.
AxdoOutPulseOn	Keep signal On at Offset location of the designated output contact module for the set mSec and turn it Off after the time.
AxdoOutPulseOff	Keep signal Off at Offset location of the designated output contact module for the set mSec and turn it On after the time.
AxdoToggleStart	Toggle signal at Offset location of the designated output contact module with the set number and the set interval and keep the initial output state after the toggle.
AxdoToggleStop	Stop output toggled at Offset location of the designated output contact module in the set signal state.

Define Sentence

Statements which are used in AXD Digital Input/Output Library are in the following list.

Header File: #include "AXHS.h"

FALSE & TRUE

Definition	Value	Explanation
FALSE	00h	False
TRUE	01h	True

AXT_USE

Definition	Value	Explanation
DISABLE	00h	Not used
ENABLE	01h	Used

AXT_EXISTENCE

Definition	Value	Explanation
STATUS_NOTEXIST	00h	Module does not exist
STATUS_EXIST	01h	Module exists

AXT_DIO_EDGE

Definition	Value	Explanation
DOWN_EDGE	00h	Falling Edge Signal
UP_EDGE	01h	Rising Edge Signal

AXT_DIO_STATE

Definition	Value	Explanation
OFF_STATE	00h	Inactive status of signal
ON_STATE	01h	Active status of signal

DIO Command Function List

Board and Module Information

Function	Description
AxdInfoIsDIOModule	Check the existence of DIO module.
AxdInfoGetModuleNo	Check the number of AIO module.
AxdInfoGetModuleCount	Check the number of DIO input/output module.
AxdInfoGetInputCount	Check the number of input contact of the designated module.
AxdInfoGetOutputCount	Check the number of output contact of the designated module.
AxdInfoGetModule	Check base board number, module location and module ID by the designated module number.

AxdInfoIsDIOModule

Purpose

Check the existence of DIO module.

Format

C++

```
DWORD AxdInfoIsDIOModule(DWORD *upStatus);
```

Visual Basic

```
Function AxdInfoIsDIOModule(ByRef upStatus As Long) As Long
```

Delphi

```
function AxdInfoIsDIOModule(upStatus : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]upStatus	Verify whether DIO module exist or not: AXT_EXISTENCE – [00h] No DIO module – [01h] DIO module exists

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

Initialize all DIO modules mounted on PC.

Example

C++

```
#include "AXL.h"        // Add library.
#include "AXHS.h"
#include "AXD.h"
DWORD dwStatus;
// Verify whether DIO module exist or not.
AxdInfoIsDIOModule(&dwStatus);
if(dwStatus == STATUS_EXIST)
   AfxMessageBox("DIO module exists.");
else
   AfxMessageBox("DIO module does not exist.");
```

Visual Basic

```
' Add AXD.bas, AXL.bas, and AXHS.bas module to use library.
Dim lStatus As Long
' Check the existence of DIO module.
```

```
AxdInfoIsDIOModule lStatus
If lStatus = STATUS_EXIST Then
  MsgBox " DIO module exists.", vbOKCancel
Else
  MsgBox " DIO module does not exist.", vbOKCancel
End If
```

Delphi

```
Uses
AXL, AXHS, AXD   { Add library. }
var
  dwStatus : DWord;

{ Verify whether DIO module exist or not. }
begin
  AxdInfoIsDIOModule(@dwStatus);
  if(dwStatus = STATUS_EXIST) then
     Application.MessageBox ('DIO module exists.', 'Ajinextek', MB_OK)
  else
     Application.MessageBox ('DIO module does not exist.', 'Ajinextek', MB_OK);
end;
```

See Also

AxdInfoGetModuleNo, AxdInfoGetModuleCount, AxdInfoGetInputCount, AxdInfoGetOutputCount, AxdInfoGetModule

AxdInfoGetModuleNo

Purpose

Check the number of DIO module.

Format

C++

```
DWORD AxdInfoGetModuleNo(long lBoardNo, long lModulePos, long
*lpModuleNo);
```

Visual Basic

```
Function AxdInfoGetModuleNo(ByVal lBoardNo As Long, ByVal lModulePos
As Long, ByRef lpModuleNo As Long) As Long
```

Delphi

```
function AxdInfoGetModuleNo(lBoardNo: LongInt; lModulePos: LongInt;
lpModuleNo: PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lBoardNo	Board Number
[>>]lModulePos	Module Position
[<<]lpModuleNo	DIO Module Number

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1101] AXT_RT_INVALID_BOARD_NO : Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS : Invalid module position
* See error code Table for more information on status error codes

Description

Check the number of DIO module initialized.

Example

C++

```
// Check the number of DIO module.
Long lpModuleNo;
CString strData;

AxdInfoGetModuleNo (0, 0, &lpModuleNo);
strData.Format("The number of DIO module is %d.", lpModuleNo);
AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of DIO module.
Dim lpModuleNo As Long
Dim  strData As String

AxdInfoGetModuleNo 0 0 lCount
strData = "The number of AIO module is" + CStr(lpModuleNo) + "."

MsgBox strData
```

Delphi

```
{ Check the number of DIO module. }
var
  lpModuleNo: LongInt;
  strData : String;

begin
  AxdInfoGetModuleNo (0, 0, @ lpModuleNo);
  strData  := 'The number of DIO module is' + IntToStr(lpModuleNo) + '.';

  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxdInfoIsDIOModule, AxdInfoGetModuleCount, AxdInfoGetInputCount, AxdInfoGetOutputCount, AxdInfoGetModule

AxdInfoGetModuleCount

Purpose

Check the number of input/output module.

Format

C++

```
DWORD AxdInfoGetModuleCount(long *lpModuleCount);
```

Visual Basic

```
Function AxdInfoGetModuleCount(ByRef lpModuleCount As Long) As Long
```

Delphi

```
function AxdInfoGetModuleCount(lpModuleCount : PLongInt) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpModuleCount	The number of DIO input/output module

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
* See error code Table for more information on status error codes

Description

Check the number of input/output modules that have been initialized.

Example

C++

```
// Check the number of DIO module.
long lCount;
CString strData;

AxdInfoGetModuleCount(&lCount);
strData.Format("The number of DIO module is %d.", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of DIO module.
Dim lCount As Long
Dim  strData As String

AxdInfoGetModuleCount lCount
```

```
strData = "The number of DIO module is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the number of DIO module. }
var
   lCount : LongInt;
   strData : String;

begin
   AxdInfoGetModuleCount(@lCount);
   strData  := 'The number of DIO module is' + IntToStr(lCount) + '.';

   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxdInfoIsDIOModule, AxdInfoGetModuleNo, AxdInfoGetInputCount, AxdInfoGetOutputCount, AxdInfoGetModule

AxdInfoGetInputCount

Purpose

Check the number of input contact of the designated module.

Format

C++

```
DWORD AxdInfoGetInputCount(long lModuleNo, long *lpCount);
```

Visual Basic

```
Function AxdInfoGetInputCount(ByVal lModuleNo As Long, ByRef lpCount
As Long) As Long
```

Delphi

```
function AxdInfoGetInputCount(lModuleNo : LongInt; lpCount :
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No (0 ~ n-1)
[<<]lpCount	Number of input contact

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[1053] AXT_RT_NOT_OPEN : Failed to initialize the library
* See error code Table for more information on status error codes

Description

Check the number of input contact of the module that user designates.

The number of input/output contact of DIO module

Module	Input	Output
SIO-DI32	32	0
SIO-DO32P	0	32
SIO-DB32P	16	16
SIO-DO32T	0	32
SIO-DB32T	16	16

Example

C++

```
// Check the number of input contact of module 0.
long lCount;
CString strData;

AxdInfoGetInputCount(0, &lCount);
strData.Format("The number of input contact of module 0 is %d.", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the number of input contact of module 0.
Dim lCount As Long
Dim  strData As String

AxdInfoGetInputCount 0, lCount
strData = "The number of input contact of module 0" is CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the number of input contact of module 0. }
var
   lCount : LongInt;
   strData : String;

begin
   AxdInfoGetInputCount(@lCount);
   strData  := 'The number of input contact of module 0 is' + IntToStr(lCount) + '.';

   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxdInfoIsDIOModule, AxdInfoGetModuleNo, AxdInfoGetModuleCount, AxdInfoGetOutputCount, AxdInfoGetModule

AxdInfoGetOutputCount

Purpose

Check the number of output contact of the designated module.

Format

C++

```
DWORD AxdInfoGetOutputCount(long lModuleNo, long *lpCount);
```

Visual Basic

```
Function AxdInfoGetOutputCount(ByVal lModuleNo As Long, ByRef lpCount
As Long) As Long
```

Delphi

```
function AxdInfoGetOutputCount(lModuleNo : LongInt; lpCount :
PLongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No (0 ~ n-1)
[<<]lpCount	Number of output contact

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
* See error code Table for more information on status error codes

Description

Check the number of output contact of the module that user designates.

Number of input/output contact of DIO module

Module	Input	Output
SIO-DI32	32	0
SIO-DO32P	0	32
SIO-DB32P	16	16
SIO-DO32T	0	32
SIO-DB32T	16	16

Example

C++

```
// Check the output contact of module 0.
long lCount;
CString strData;

AxdInfoGetOutputCount(0, &lCount);
strData.Format("The number of output contact of module 0 is %d.", lCount);

AfxMessageBox(strData);
```

Visual Basic

```
' Check the output contact of module 0.
Dim lCount As Long
Dim  strData As String

AxdInfoGetOutputCount 0, lCount
strData = "The number of output contact of module 0 is" + CStr(lCount) + "."

MsgBox strData
```

Delphi

```
{ Check the output contact of module 0. }
var
   lCount : LongInt;
   strData : String;

begin
   AxdInfoGetOutputCount(@lCount);
   strData  := 'The number of output contact of module 0 is' + IntToStr(lCount) + '.' ;

   Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxdInfoIsDIOModule, AxdInfoGetModuleNo, AxdInfoGetModuleCount, AxdInfoGetInputCount, AxdInfoGetModule

AxdInfoGetModule

Purpose

Check base board number, module location and module ID by the designated module number.

Format

C++

```
DWORD AxdInfoGetModule(long lModuleNo, long *lpBoardNo, long
*lpModulePos, DWORD *upModuleID);
```

Visual Basic

```
Function AxdInfoGetModule(ByVal lModuleNo As Long, ByRef lpBoardNo As
Long, ByRef lpModulePos As Long, ByRef upModuleID As Long) As Long
```

Delphi

```
function AxdInfoGetModule(lModuleNo : LongInt; lpBoardNo : PLongInt;
lpModulePos : PLongInt; upModuleID : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No (0 ~ n-1)
[<<]lpBoardNo	Base board No (0 ~ N-1)
[<<]lpModulePos	Module location (0 ~ 3)
[<<]upModuleID	Module ID

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
* See error code Table for more information on status error codes

Description

Check base board number, module location and module ID by the module number user designates.

IDs of DIO module are listed as follows.

Module ID	HEX Value (Decimal)	Module
AXT_SIO_DI32	0x97 (151)	Digital IN 32
AXT_SIO_DO32P	0x98 (152)	Digital OUT 32
AXT_SIO_DB32P	0x99 (153)	Digital IN 16 / OUT 16
AXT_SIO_DO32T	0x9E (158)	Digital OUT 16, Power TR OUT
AXT_SIO_DB32T	0x9F(159)	Digital IN 16 / OUT 16, Power TR OUT
AXT_SIO_RDI32	0x95	Digital IN 32

AXT_SIO_RDO32	0x96	Digital OUT 32
AXT_SIO_RDB128MLII	0x94	Digital IN 64 / OUT 64

Example

C++

```
// Check base board No., module No., and module ID of module 0.
long lBoardNo;
long lModulePos;
DWORD dwModuleID;
CString strData;
AxdInfoGetModule(0, &lBoardNo, &lModulePos, &dwModuleID);
strData.Format("BoardNo=%d, ModulePos=%d, dwModuleID=%02Xh", lBoardNo, lModulePos,
  dwModuleID);
AfxMessageBox(strData);
```

Visual Basic

```
' Check base board No., module No., and module ID of module 0.
Dim lBoardNo As Long
Dim lModulePos As Long
Dim dwModuleID As DWord
Dim strData As String
AxdInfoGetModule 0, lBoardNo, lModulePos, dwModuleID
strData = "BoardNo=" + CStr(lBoardNo) + ", ModulePos=" + CStr(lModulePos) + ",
  ModuleID=" + CStr(dwModuleID)
MsgBox strData
```

Delphi

```
{ Check base board No, module No, and module ID of module 0. }
var
  BoardNo : LongInt;
  lModulePos : LongInt;
  dwModuleID : DWord;
  strData : String;

begin
  AxdInfoGetModule(0, @lBoardNo, @lModulePos, @dwModuleID);
  strData := 'BoardNo=' + IntToStr(lBoardNo) + ', ModulePos=' + IntToStr(lModulePos) +
     ', ModuleID=' + IntToStr(dwModuleID);
  Application.MessageBox (PCHAR(strData), 'Ajinextek', MB_OK);
end;
```

See Also

AxdInfoIsDIOModule, AxdInfoGetModuleNo, AxdInfoGetModuleCount, AxdInfoGetInputCount, AxdInfoGetOutputCount

Interrupt

Checking the setting of Interrupt

Function	Description
AxdiInterruptSetModule	Use window message, call back API, or event method to bring Interrupt message to the designated module.
AxdiInterruptSetModuleEnable	Set whether to make the designated module use Interrupt.
AxdiInterruptGetModuleEnable	Check whether the designated module uses Interrupt.
AxdiInterruptRead	Check the location where Interrupt is created.

Checking the setting of Interrupt rising/falling edge

Function	Description
AxdiInterruptEdgeSetBit	Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSetByte	Set Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSetWord	Set Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSetDword	Set Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetBit	Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetByte	Check Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetWord	Check Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeGetDword	Check Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.
AxdiInterruptEdgeSet	Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.
AxdiInterruptEdgeGet	Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.

AxdiInterruptSetModule

Purpose

Use window message, call back API, or event method to bring Interrupt message to the designated module.

Format

C++

```
DWORD AxdiInterruptSetModule(long lModuleNo, HWND hWnd, DWORD
uMessage, AXT_INTERRUPT_PROC pProc, HANDLE *pEvent);
```

Visual Basic

```
Function AxdiInterruptSetModule(ByVal lModuleNo As Long, ByVal hWnd
As Long, ByVal uMessage As Long, ByVal pProc As Long, ByRef pEvent As
Long) As Long
```

Delphi

```
function AxdiInterruptSetModule(lModuleNo : LongInt; hWnd : HWND;
uMessage : DWord; pProc : AXT_INTERRUPT_PROC; pEvent : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No (0 ~ N-1)
[>>]hWnd	Window handle
[>>]uMessage	Window message
[>>]pProc	Call back API
[<<]pEvent	Event ID

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
* See error code Table for more information on status error codes

Description

Interrupt can be received in the unit of module.

To receive Interrupt message, return window handle and when using call back API, return API pointer.

When using event method, bring event ID to receive Interrupt.

For argument, refer to the table below.

Argument	Description

lModuleNo	Module number that would receive Interrupt
hwnd	Window handle. Use when receiving window message. Set as NULL if not in use
uMessage	Window handle message. Set it as 0 if not in use or default value is used
Proc	Pointer of an API that will be called up when Interrupt is created. Set as NULL if not in use
pEvent	Event ID. Use for the case of event method. Set as NULL if not in use

Example

C++

```
// Use window message to bring Interrupt message to module 0.
// Add to Header File.
afx_msg LRESULT OnInterruptMessage (WPARAM wParam, LPARAM lParam) ;

BEGIN_MESSAGE_MAP(CDIODlg, CDialog)
   //{{AFX_MSG_MAP
   // Declaration
   ON_MESSAGE(WM_AXL_INTERRUPT, OnInterruptMessage)
   //}}AFX_MSG_MAP
END_MESSAGE_MAP()

// When Interrupt is created, 'OnInterruptMessage()' is called up.
AxdiInterruptSetModule(0, m_hWnd, WM_AXL_INTERRUPT, NULL, NULL);
```

Visual Basic

```
' Use window message to bring Interrupt message to module 0.
' Declaration
Private Sub AxtMsg1_OnMessage1(ByVal wParam As Long, ByVal lParam As Long)
' Interrupt message process syntax
End Sub

' When Interrupt is created, 'AxtMsg1_OnMessage1()' is called up.
AxtMsg1.Message1 = WM_AXL_INTERRUPT

AxdiInterruptSetModule 0, Me.hWnd, WM_AXL_INTERRUPT, 0, 0
```

Delphi

```
{ Use window message to bring Interrupt message to module 0. }
{ Declaration }
procedure TForm1.OnInterruptMessage(var Msg : TMessage);
begin
{ Interrupt message process syntax }
end;
{ When Interrupt is created, 'OnInterruptMessage()' is called up. }
AxdiInterruptSetModule(0, Form1.Handle, WM_AXL_INTERRUPT, nil, nil);
```

See Also

AxdiInterruptSetModuleEnable, AxdiInterruptGetModuleEnable, AxdiInterruptRead

AxdiInterruptSetModuleEnable

Purpose

Set whether to make the designated module use Interrupt.

Format

C++

```
DWORD AxdiInterruptSetModuleEnable(long lpModuleNo, DWORD uUse);
```

Visual Basic

```
Function AxdiInterruptSetModuleEnable(ByVal lpModuleNo As Long, ByVal
uUse As Long) As Long
```

Delphi

```
function AxdiInterruptSetModuleEnable(lpModuleNo : LongInt; uUse :
DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lpModuleNo	Module No (0 ~ n-1)
[>>]uUse	Set whether to use interrupt : AXT_USE - [00h] Release Interrupt - [01h] Set Interrupt

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
* See error code Table for more information on status error codes

Description

Set whether to make the module that user designates use Interrupt.

Example

C++

```
// Set to make Interrupt of module 0 usable.
AxdiInterruptSetModuleEnable(0, ENABLE);
```

Visual Basic

```
' Set to make Interrupt of module 0 usable.
AxdiInterruptSetModuleEnable 0, ENABLE
```

Delphi

```
{ Set to make Interrupt of module 0 usable. }
AxdiInterruptSetModuleEnable(0, ENABLE);
```

See Also

AxdiInterruptSetModule, AxdiInterruptGetModuleEnable, AxdiInterruptRead

AxdiInterruptGetModuleEnable

Purpose

Check whether the designated module uses Interrupt.

Format

C++

```
DWORD AxdiInterruptGetModuleEnable(long lpModuleNo, DWORD *upUse);
```

Visual Basic

```
Function AxdiInterruptGetModuleEnable(ByVal lpModuleNo As Long, ByRef
upUse As Long) As Long
```

Delphi

```
function AxdiInterruptGetModuleEnable(lpModuleNo : LongInt; upUse :
PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lpModuleNo	Module No (0 ~ n-1)
[<<]upUse	Set whether to use interrupt : AXT_USE - [00h] Release Interrupt - [01h] Set Interrupt

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
* See error code Table for more information on status error codes

Description

Check whether the module that user designates uses Interrupt.

Example

C++

```
// Check whether module 0 can use Interrupt.
DWORD dwUse;
AxdiInterruptGetModuleEnable(0, &dwUse);
if (dwUse)
{
  AfxMessageBox("The module 0 can use Interrupt.");
}
else
{
  AfxMessageBox("The module 0 cannot use Interrupt.");
}
```

Visual Basic

```
' Check whether module 0 can use Interrupt.
Dim lUse As Long

AxdiInterruptGetModuleEnable 0, lUse

If lUse = ENABLE Then
  MsgBox "Module 0 can use Interrupt.", vbOKCancel
Else
  MsgBox "Module 0 cannot use Interrupt.", vbOKCancel
End If
```

Delphi

```
{ Check whether module 0 can use Interrupt. }
var
  dwUse : DWord;

begin
  AxdiInterruptGetModuleEnable(0, @dwUse);
  if (dwUse = ENABLE) then
     Application.MessageBox ('Module 0 can use Interrupt.', 'Ajinextek', MB_OK);
  else
     Application.MessageBox ('Module 0 cannot use Interrupt.', 'Ajinextek', MB_OK);
end;
```

See Also

AxdiInterruptSetModule, AxdiInterruptSetModuleEnable, AxdiInterruptRead

AxdiInterruptRead

Purpose

Check the location where Interrupt is created.

Format

C++

```
DWORD AxdiInterruptRead(long *lpModuleNo, DWORD *upFlag);
```

Visual Basic

```
Function AxdiInterruptRead(ByRef lpModuleNo As Long, ByRef upFlag As
Long) As Long
```

Delphi

```
function AxdiInterruptRead(lpModuleNo : PLongInt; upFlag : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[<<]lpModuleNo	Module No
[<<]upFlag	Location where Interrupt is created

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3052] AXT_RT_DIO_NOT_INTERRUPT: DIO Interrupt not set
* See error code Table for more information on status error codes

Description

Check the location where Interrupt is created when using event method for Interrupt.

Example

C++

```
// Check the location in module 0 where Interrupt is created.
DWORD dwFlag;

AxdiInterruptRead(0, &dwFlag);
```

Visual Basic

```
' Check the location in module 0 where Interrupt is created.
Dim lFlag As long

AxdiInterruptRead 0, lFlag
```

Delphi

```
{ Check the location in module 0 where Interrupt is created. }
var
   dwFlag : DWord;

begin
   AxdiInterruptRead(0, @dwFlag);
end;
```

See Also

AxdiInterruptSetModule, AxdiInterruptSetModuleEnable, AxdiInterruptGetModuleEnable

AxdiInterruptEdgeSetBit

Purpose

Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeSetBit(long lModuleNo, long lOffset, DWORD uMode, DWORD uValue);
```

Visual Basic

```
Function AxdiInterruptEdgeSetBit(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeSetBit(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location form input contact
[>>]uMode	Set interrupt : AXT_DIO_EDGE – [00h] Falling edge – [01h] Rising edge
[>>]uValue	The state whether Interrupt creation is set. (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0 ~ 31	0(Off), 1(On)
SIO-DB32x	0 ~ 15	0(Off), 1(On)

Example

C++

```
// Set Rising edge value in bit unit at Offset location of Interrupt Rising Edge
// Register in the input contact module 0.
AxdiInterruptEdgeSetBit(0, 0, UP_EDGE, 1);
```

Visual Basic

```
' Set Rising edge value in bit unit at Offset location of Interrupt Rising Edge Register
' in the input contact module 0.
AxdiInterruptEdgeSetBit 0, 0, UP_EDGE, 1
```

Delphi

```
{ Set Rising edge value in bit unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
AxdiInterruptEdgeSetBit(0, 0, UP_EDGE, 1);
```

See Also

AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte, AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeSetByte

Purpose

Set Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeSetByte(long lModuleNo, long lOffset, DWORD uMode, DWORD uValue);
```

Visual Basic

```
Function AxdiInterruptEdgeSetByte(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeSetByte(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location form input contact
[>>]uMode	Set interrupt : AXT_DIO_EDGE – [00h] Falling edge – [01h] Rising edge
[>>]uValue	The state whether Interrupt creation is set. (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Set Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0, 1, 2, 3	00h ~ FFh
SIO-DB32x	0, 1	00h ~ FFh

Example

C++

```
// Set Rising edge value in byte unit at Offset location of Interrupt Rising Edge
// Register in the input contact module 0.
AxdiInterruptEdgeSetByte(0, 0, UP_EDGE, 0xFF);
```

Visual Basic

```
' Set Rising edge value in byte unit at Offset location of Interrupt Rising Edge
' Register in the input contact module 0.
AxdiInterruptEdgeSetByte 0, 0, UP_EDGE, &hFF
```

Delphi

```
{ Set Rising edge value in byte unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
AxdiInterruptEdgeSetByte(0, 0, UP_EDGE, $FF);
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte, AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeSetWord

Purpose

Set Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeSetWord(long lModuleNo, long lOffset, DWORD uMode, DWORD uValue);
```

Visual Basic

```
Function AxdiInterruptEdgeSetWord(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeSetWord(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uMode	Set interrupt : AXT_DIO_EDGE – [00h] Falling edge – [01h] Rising edge
[>>]uValue	The state whether Interrupt creation is set. (word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Set Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0, 1	0000h ~ FFFFh
SIO-DB32x	0	0000h ~ FFFFh

Example

C++

```
// Set Rising edge value in word unit at Offset location of Interrupt Rising Edge
// Register in the input contact module 0.
AxdiInterruptEdgeSetWord(0, 0, UP_EDGE, 0xFFFF);
```

Visual Basic

```
' Set Rising edge value in word unit at Offset location of Interrupt Rising Edge
' Register in the input contact module 0.
AxdiInterruptEdgeSetWord 0, 0, UP_EDGE, &hFFFF
```

Delphi

```
{ Set Rising edge value in word unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
AxdiInterruptEdgeSetWord(0, 0, UP_EDGE, $FFFF);
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte, AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeSetDword

Purpose

Set Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeSetDword(long lModuleNo, long lOffset, DWORD uMode, DWORD uValue);
```

Visual Basic

```
Function AxdiInterruptEdgeSetDword(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeSetDword(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uMode	Set interrupt : AXT_DIO_EDGE – [00h] Falling edge – [01h] Rising edge
[>>]uValue	The state whether Interrupt creation is set. (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Set Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0	00000000h ~ FFFFFFFFh
SIO-DB32x	0	00000000h ~ 0000FFFFh

Example

C++

```
// Set Rising edge value in double word unit at Offset location of Interrupt Rising
// Edge Register in the input contact module 0.
AxdiInterruptEdgeSetDword(0, 0, UP_EDGE, 0xFFFFFFFF);
```

Visual Basic

```
' Set Rising edge value in double word unit at Offset location of Interrupt Rising Edge
' Register in the input contact module 0.
AxdiInterruptEdgeSetDword 0, 0, UP_EDGE, &hFFFFFFFF
```

Delphi

```
{ Set Rising edge value in double word unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
AxdiInterruptEdgeSetDword(0, 0, UP_EDGE, $FFFFFFFF);
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte, AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeGetBit

Purpose

Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeGetBit(long lModuleNo, long lOffset, DWORD uMode, DWORD *upValue);
```

Visual Basic

```
Function AxdiInterruptEdgeGetBit(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeGetBit(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uMode	Interrupt setting : AXT_DIO_EDGE – [00h] Falling edge – [01h] Rising edge
[<<]upValue	The state whether Interrupt creation is set. (bit)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0 ~ 31	0(Off), 1(On)
SIO-DB32x	0 ~ 15	0(Off), 1(On)

Example

C++

```
// Read Rising edge set value in bit unit at Offset location of Interrupt Rising Edge
// Register in the input contact module 0.
DWORD dwEdge

AxdiInterruptEdgeGetBit(0, 0, UP_EDGE, &dwEdge);
```

Visual Basic

```
' Read Rising edge set value in bit unit at Offset location of Interrupt Rising Edge
' Register in the input contact module 0.
Dim lEdge As long

AxdiInterruptEdgeGetBit 0, 0, UP_EDGE, lEdge
```

Delphi

```
{ Read Rising edge set value in bit unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
var
  dwEdge : DWord;

begin
  AxdiInterruptEdgeGetBit(0, 0, UP_EDGE, @dwEdge);
end;
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetByte, AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeGetByte

Purpose

Check Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeGetByte(long lModuleNo, long lOffset, DWORD uMode, DWORD *upValue);
```

Visual Basic

```
Function AxdiInterruptEdgeGetByte(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeGetByte(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uMode	Interrupt setting : AXT_DIO_EDGE – [00h] Falling edge – [01h] Rising edge
[<<]upValue	The state whether Interrupt creation is set. (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Check Rising or Falling edge value in byte unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0, 1, 2, 3	00h ~ FFh
SIO-DB32x	0, 1	00h ~ FFh

Example

C++

```
// Read Rising edge set value in byte unit at Offset location of Interrupt Rising Edge
// Register in the input contact module 0.
DWORD dwEdge

AxdiInterruptEdgeGetByte(0, 0, UP_EDGE, &dwEdge);
```

Visual Basic

```
' Read Rising edge set value in byte unit at Offset location of Interrupt Rising Edge
' Register in the input contact module 0.
Dim lEdge As long

AxdiInterruptEdgeGetByte 0, 0, UP_EDGE, lEdge
```

Delphi

```
{ Read Rising edge set value in byte unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
var
  dwEdge : DWord;

begin
  AxdiInterruptEdgeGetByte(0, 0, UP_EDGE, @dwEdge);
end;
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeGetWord

Purpose

Check Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeGetWord(long lModuleNo, long lOffset, DWORD uMode, DWORD *upValue);
```

Visual Basic

```
Function AxdiInterruptEdgeGetWord(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeGetWord(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uMode	Interrupt setting : AXT_DIO_EDGE - [00h] Falling edge - [01h] Rising edge
[<<]upValue	The state whether Interrupt creation is set. (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Check Rising or Falling edge value in word unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value

SIO-DI32	0, 1	0000h ~ FFFFh
SIO-DB32x	0	0000h ~ FFFFh

Example

C++

```
// Read Rising edge set value in word unit at Offset location of Interrupt Rising Edge
// Register in the input contact module 0.
DWORD dwEdge

AxdiInterruptEdgeGetWord(0, 0, UP_EDGE, &dwEdge);
```

Visual Basic

```
' Read Rising edge set value in word unit at Offset location of Interrupt Rising Edge
' Register in the input contact module 0.
Dim lEdge As long

AxdiInterruptEdgeGetWord 0, 0, UP_EDGE, lEdge
```

Delphi

```
{ Read Rising edge set value in word unit at Offset location of Interrupt Rising Edge
Register in the input contact module 0. }
var
  dwEdge : DWord;

begin
  AxdiInterruptEdgeGetWord(0, 0, UP_EDGE, @dwEdge);
end;
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeGetDword

Purpose

Check Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the designated input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeGetDword(long lModuleNo, long lOffset, DWORD uMode, DWORD *upValue);
```

Visual Basic

```
Function AxdiInterruptEdgeGetDword(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uMode As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeGetDword(lModuleNo : LongInt; lOffset : LongInt; uMode : DWord; upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uMode	Interrupt setting : AXT_DIO_EDGE - [00h] Falling edge - [01h] Rising edge
[<<]upValue	The state whether Interrupt creation is set. (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Check Rising or Falling edge value in double word unit at Offset location of Interrupt Rising / Falling Edge Register in the input contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Return value

Module	Offset	Value

SIO-DI32	0	00000000h ~ FFFFFFFFh
SIO-DB32x	0	00000000h ~ 0000FFFFh

Example

C++

```
// Read Rising edge set value in double word unit at Offset location of Interrupt
// Rising Edge Register in the input contact module 0.
DWORD dwEdge

AxdiInterruptEdgeGetDword(0, 0, UP_EDGE, &dwEdge);
```

Visual Basic

```
' Read Rising edge set value in double word unit at Offset location of Interrupt Rising
' Edge Register in the input contact module 0.
Dim lEdge As long

AxdiInterruptEdgeGetDword 0, 0, UP_EDGE, lEdge
```

Delphi

```
{ Read Rising edge set value in double word unit at Offset location of Interrupt Rising
Edge Register in the input contact module 0. }
var
  dwEdge : DWord;

begin
  AxdiInterruptEdgeGetDword(0, 0, UP_EDGE, @dwEdge);
end;
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte AxdiInterruptEdgeGetWord, AxdiInterruptEdgeSet, AxdiInterruptEdgeGet

AxdiInterruptEdgeSet

Purpose

Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeSet(long lOffset, DWORD uMode, DWORD uValue);
```

Visual Basic

```
Function AxdiInterruptEdgeSet(ByVal lOffset As Long, ByVal uMode As
Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeSet(lOffset : LongInt; uMode : DWord;
uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from input contact
[>>]uMode	Interrupt setting : AXT_DIO_EDGE - [00h] Falling edge - [01h] Rising edge
[>>]uValue	The state whether Interrupt creation is set. (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.

Offset ranges from 0 to the number of total input contact-1.

Example

C++

```
// Set Rising or Falling edge value in bit unit at Offset location of Interrupt
// Rising / Falling Edge Register in the entire input contact module.
AxdiInterruptEdgeSet(0, UP_EDGE, 1);
```

Visual Basic

```
' Set Rising or Falling edge value in bit unit at Offset location of Interrupt
' Rising / Falling Edge Register in the entire input contact module.
AxdiInterruptEdgeSet 0, UP_EDGE, 1
```

Delphi

```
{ Set Rising or Falling edge value in bit unit at Offset location of Interrupt Rising /
Falling Edge Register in the entire input contact module. }
AxdiInterruptEdgeSet(0, UP_EDGE, 1);
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeGet

AxdiInterruptEdgeGet

Purpose

Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.

Format

C++

```
DWORD AxdiInterruptEdgeGet(long lOffset, DWORD uMode, DWORD *upValue);
```

Visual Basic

```
Function AxdiInterruptEdgeGet(ByVal lOffset As Long, ByVal uMode As
Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiInterruptEdgeGet(lOffset : LongInt; uMode : DWord;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from input contact
[>>]uMode	Interupt setting : AXT_DIO_EDGE - [00h] Falling edge - [01h] Rising edge
[<<]upValue	The state whether Interrupt creation is set. (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
[3104] AXT_RT_DIO_INVALID_MODE : Invalid DIO mode
* See error code Table for more information on status error codes

Description

Check Rising or Falling edge value in bit unit at Offset location of Interrupt Rising / Falling Edge Register in the entire input contact module.

Offset ranges from 0 to the number of total input contact-1.

Example

C++

```
// Set Rising edge value in bit unit at Offset location of Interrupt Rising Edge
// Register in the entire input module.
DWORD dwEdge

AxdiInterruptEdgeGet(0, UP_EDGE, &dwEdge);
```

Visual Basic

```
' Set Rising edge value in bit unit at Offset location of Interrupt Rising Edge Register
' in the entire input module.
Dim lEdge As long

AxdiInterruptEdgeGet 0, UP_EDGE, lEdge
```

Delphi

```
{ Set Rising edge value in bit unit at Offset location of Interrupt Rising Edge
Register in the entire input module. }
var
   dwEdge : DWord;

begin
   AxdiInterruptEdgeGet(0, UP_EDGE, @dwEdge);
end;
```

See Also

AxdiInterruptEdgeSetBit, AxdiInterruptEdgeSetByte, AxdiInterruptEdgeSetWord, AxdiInterruptEdgeSetDword, AxdiInterruptEdgeGetBit, AxdiInterruptEdgeGetByte AxdiInterruptEdgeGetWord, AxdiInterruptEdgeGetDword, AxdiInterruptEdgeSet

Checking the setting of input level

Function	Description
AxdiLevelSetInportBit	Set data level in bit unit at Offset location of the designated input contact module.
AxdiLevelSetInportByte	Set data level in byte unit at Offset location of the designated input contact module.
AxdiLevelSetInportWord	Set data level in word unit at Offset location of the designated input contact module.
AxdiLevelSetInportDword	Set data level in double word unit at Offset location of the designated input contact module.
AxdiLevelGetInportBit	Check data level in bit unit at Offset location of the designated input contact module.
AxdiLevelGetInportByte	Check data level in byte unit at Offset location of the designated input contact module.
AxdiLevelGetInportWord	Check data level in word unit at Offset location of the designated input contact module.
AxdiLevelGetInportDword	Check data level in double word unit at Offset location of the designated input contact module.
AxdiLevelSetInport	Set data level in bit unit at Offset locations of the entire input contact module.
AxdiLevelGetInport	Check data level in bit unit at Offset locations of the entire input contact module.

AxdiLevelSetInportBit

Purpose

Set data level in bit unit at Offset location of the designated input contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelSetInportBit(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdiLevelSetInportBit(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdiLevelSetInportBit(lModuleNo : LongInt; lOffset : LongInt;
uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in bit unit at Offset location of the designated input contact module.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0 ~ 31	0(LOW), 1(HIGH)
SIO-DO32P	-	-
SIO-DB32P	0 ~ 15	0(LOW), 1(HIGH)

SIO-DO32T	-	-
SIO-DB32T	0 ~ 15	0(LOW), 1(HIGH)

Example

C++

```
// Set data level in bit unit at Offset 0 in module 0.
AxdiLevelSetInportBit(0, 0, 1);
```

Visual Basic

```
' Set data level in bit unit at Offset 0 in module 0.
AxdiLevelSetInportBit 0, 0, 1
```

Delphi

```
{ Set data level in bit unit at Offset 0 in module 0. }
AxdiLevelSetInportBit(0, 0, 1);
```

See Also

AxdiLevelSetInportByte, AxdiLevelSetInportWord, AxdiLevelSetInportDword, AxdiLevelSetInport

AxdiLevelSetInportByte

Purpose

Set data level in byte unit at Offset location of the designated input contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelSetInportByte(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdiLevelSetInportByte(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdiLevelSetInportByte(lModuleNo : LongInt; lOffset :
LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uLevel	Level value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in byte unit at Offset location of the input contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0, 1, 2, 3	00h ~ FFh
SIO-DO32P	-	-
SIO-DB32P	0, 1	00h ~ FFh

SIO-DO32T	-	-
SIO-DB32T	0, 1	00h ~ FFh

Example

C++

```
// Set data level in byte unit at Offset 0 in module 0.
AxdiLevelSetInportByte(0, 0, 0xFF);
```

Visual Basic

```
' Set data level in byte unit at Offset 0 in module 0.
AxdiLevelSetInportByte 0, 0, &hFF
```

Delphi

```
{ Set data level in byte unit at Offset 0 in module 0. }
AxdiLevelSetInportByte(0, 0, $FF);
```

See Also

AxdiLevelSetInportBit, AxdiLevelSetInportWord, AxdiLevelSetInportDword, AxdiLevelSetInport

AxdiLevelSetInportWord

Purpose

Set data level in word unit at Offset location of the designated input contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelSetInportWord(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdiLevelSetInportWord(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdiLevelSetInportWord(lModuleNo : LongInt; lOffset :
LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uLevel	Level value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in word unit at Offset location of the input contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0, 1	0000h ~ FFFFh
SIO-DO32P	-	-
SIO-DB32P	0	0000h ~ FFFFh

SIO-DO32T	-	-
SIO-DB32T	0	0990h ~ FFFFh

Example

C++

```
// Set data level in word unit at Offset 0 in module 0.
AxdiLevelSetInportWord(0, 0, 0xFFFF);
```

Visual Basic

```
' Set data level in word unit at Offset 0 in module 0.
AxdiLevelSetInportWord 0, 0, &hFFFF
```

Delphi

```
{ Set data level in word unit at Offset 0 in module 0. }
AxdiLevelSetInportWord(0, 0, $FFFF);
```

See Also

AxdiLevelSetInportBit, AxdiLevelSetInportByte, AxdiLevelSetInportDword, AxdiLevelSetInport

AxdiLevelSetInportDword

Purpose

Set data level in double word unit at Offset location of the designated input contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelSetInportDword(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdiLevelSetInportDword(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdiLevelSetInportDword(lModuleNo : LongInt; lOffset :
LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uLevel	Level value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in double word unit at Offset location of the designated input contact module.

Check which module is the designated module and set the value with referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0,	00000000h ~ FFFFFFFFh
SIO-DO32P	-	-
SIO-DB32P	0	00000000h ~ FFFFFFFFh

SIO-DO32T	-	-
SIO-DB32T	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Set data level in double word unit at Offset 0 in module 0.
AxdiLevelSetInportDword(0, 0, 0xFFFFFFFF);
```

Visual Basic

```
' Set data level in double word unit at Offset 0 in module 0.
AxdiLevelSetInportDword 0, 0, &hFFFFFFFF
```

Delphi

```
{ Set data level in double word unit at Offset 0 in module 0. }
AxdiLevelSetInportDword(0, 0, $FFFFFFFF);
```

See Also

AxdiLevelSetInportBit, AxdiLevelSetInportByte, AxdiLevelSetInportWord, AxdiLevelSetInport

AxdiLevelGetInportBit

Purpose

Check data level in bit unit at Offset location of the designated input contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelGetInportBit(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdiLevelGetInportBit(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdiLevelGetInportBit(lModuleNo : LongInt; lOffset : LongInt;
upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in bit unit at Offset location of the input contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0 ~ 31	0(LOW), 1(HIGH)
SIO-DO32P	-	-
SIO-DB32P	0 ~ 15	0(LOW), 1(HIGH)

SIO-DO32T	-	-
SIO-DB32T	0 ~ 15	0(LOW), 1(HIGH)

Example

C++

```
// Check data level in bit unit at Offset 0 in module 0.
DWORD dwLevel;

AxdiLevelGetInportBit(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in bit unit at Offset 0 in module 0.
Dim lLevel As long

AxdiLevelGetInportBit 0, 0, lLevel
```

Delphi

```
{ Check data level in bit unit at Offset 0 in module 0. }
var
  dwLevel : DWord;

begin
  AxdiLevelGetInportBit(0, 0, @dwLevel);
end;
```

See Also

AxdiLevelGetInportByte, AxdiLevelGetInportWord, AxdiLevelGetInportDword, AxdiLevelGetInport

AxdiLevelGetInportByte

Purpose

Check data level in byte unit at Offset location of the designated input contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelGetInportByte(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdiLevelGetInportByte(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdiLevelGetInportByte(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upLevel	Level value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in byte unit at Offset location of the input contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0, 1, 2, 3	00h ~ FFh
SIO-DO32P	-	-
SIO-DB32P	0, 1	00h ~ FFh

SIO-DO32T	-	-
SIO-DB32T	0, 1	00h ~ FFh

Example

C++

```
// Check data level in byte unit at Offset 0 in module 0.
DWORD dwLevel;

AxdiLevelGetInportByte(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in byte unit at Offset 0 in module 0.
Dim lLevel As long

AxdiLevelGetInportByte 0, 0, lLevel
```

Delphi

```
{ Check data level in byte unit at Offset 0 in module 0. }
var
  dwLevel : DWord;

begin
  AxdiLevelGetInportByte(0, 0, @dwLevel);
end;
```

See Also

AxdiLevelGetInportBit, AxdiLevelGetInportWord, AxdiLevelGetInportDword, AxdiLevelGetInport

AxdiLevelGetInportWord

Purpose

Check data level in word unit at Offset location of the designated input contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelGetInportWord(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdiLevelGetInportWord(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdiLevelGetInportWord(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upLevel	Level value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in word unit at Offset location of the input contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0, 1	0000h ~ FFFFh
SIO-DO32P	-	-
SIO-DB32P	0	0000h ~ FFFFh

SIO-DO32T	-	-
SIO-DB32T	0	0990h ~ FFFFh

Example

C++

```
// Check data level in word unit at Offset 0 in module 0.
DWORD dwLevel;

AxdiLevelGetInportWord(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in word unit at Offset 0 in module 0.
Dim lLevel As long

AxdiLevelGetInportWord 0, 0, lLevel
```

Delphi

```
{ Check data level in word unit at Offset 0 in module 0. }
var
  dwLevel : DWord;

begin
  AxdiLevelGetInportWord(0, 0, @dwLevel);
end;
```

See Also

AxdiLevelGetInportBit, AxdiLevelGetInportByte, AxdiLevelGetInportDword, AxdiLevelGetInport

AxdiLevelGetInportDword

Purpose

Check data level in double word unit at Offset location of the designated input contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelGetInportDword(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdiLevelGetInportDword(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdiLevelGetInportDword(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upLevel	Level value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in double word unit at Offset location of the input contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	0,	00000000h ~ FFFFFFFFh
SIO-DO32P	-	-
SIO-DB32P	0	00000000h ~ FFFFFFFFh

SIO-DO32T	-	-
SIO-DB32T	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Check data level in double word unit at Offset 0 in module 0.
DWORD dwLevel;

AxdiLevelGetInportDword(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in double word unit at Offset 0 in module 0.
Dim lLevel As long

AxdiLevelGetInportDword 0, 0, lLevel
```

Delphi

```
{ Check data level in double word unit at Offset 0 in module 0. }
var
  dwLevel : DWord;

begin
  AxdiLevelGetInportDword(0, 0, @dwLevel);
end;
```

See Also

AxdiLevelGetInportBit, AxdiLevelGetInportByte, AxdiLevelGetInportWord, AxdiLevelGetInport

AxdiLevelSetInport

Purpose

Set data level in bit unit at Offset locations of the entire input contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelSetInport(long lOffset, DWORD uLevel);
```

Visual Basic

```
Function AxdiLevelSetInport(ByVal lOffset As Long, ByVal uLevel As
Long) As Long
```

Delphi

```
function AxdiLevelSetInport(lOffset : LongInt; uLevel : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset locaton from the entire input contact
[>>]uLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in bit unit at Offset locations of the entire input contact module.

Offset ranges from 0 to the number of total input contact-1.

Example

C++

```
// Set data level in bit unit at Offset 0 in the entire module.
AxdiLevelSetInport(0, 1);
```

Visual Basic

```
' Set data level in bit unit at Offset 0 in the entire module.
AxdiLevelSetInport 0, 1
```

Delphi

```
{ Set data level in bit unit at Offset 0 in the entire module. }
AxdiLevelSetInport(0, 1);
```

See Also

AxdiLevelSetInportBit, AxdiLevelSetInportByte, AxdiLevelSetInportWord, AxdiLevelSetInportDword, AxdiLevelGetInport

AxdiLevelGetInport

Purpose

Check data level in bit unit at Offset locations of the entire input contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdiLevelGetInport(long lOffset, DWORD *upLevel);
```

Visual Basic

```
Function AxdiLevelGetInport(ByVal lOffset As Long, ByRef upLevel As
Long) As Long
```

Delphi

```
function AxdiLevelGetInport(lOffset : LongInt; upLevel : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset locaton from the entire input contact
[<<]upLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in bit unit at Offset locations of the entire input contact module.

Offset ranges from 0 to the number of entire input contact-1.

Example

C++

```
// Check data level in bit unit at Offset 0 in the entire module.
DWORD dwLevel;

AxdiLevelGetInport(0, &dwLevel);
```

Visual Basic

```
' Check data level in bit unit at Offset 0 in the entire module.
Dim lLevel As long

AxdiLevelGetInport 0, lLevel
```

Delphi

```
{ Check data level in bit unit at Offset 0 in the entire module. }
var
   dwLevel : DWord;

begin
   AxdiLevelGetInportBit(0, @dwLevel);
end
```

See Also

AxdiLevelGetInportBit, AxdiLevelGetInportByte, AxdiLevelGetInportWord, AxdiLevelGetInportDword, AxdiLevelSetInport

Checking the setting of output level

Function	Description
AxdoLevelSetOutportBit	Set data level in bit unit at Offset location of the designated output contact module.
AxdoLevelSetOutportByte	Set data level in byte unit at Offset location of the designated output contact module.
AxdoLevelSetOutportWord	Set data level in word unit at Offset location of the designated output contact module.
AxdoLevelSetOutportDword	Set data level in double word unit at Offset location of the designated output contact module.
AxdoLevelGetOutportBit	Check data level in bit unit at Offset location of the designated output contact module.
AxdoLevelGetOutportByte	Check data level in byte unit at Offset location of the designated output contact module.
AxdoLevelGetOutportWord	Check data level in word unit at Offset location of the designated output contact module.
AxdoLevelGetOutportDword	Check data level in double word unit at Offset location of the designated output contact module.
AxdoLevelSetOutport	Set data level in bit unit at Offset locations of the entire output contact module.
AxdoLevelGetOutport	Check data level in bit unit at Offset locations of the entire output contact module.

AxdoLevelSetOutportBit

Purpose

Set data level in bit unit at Offset location of the designated output contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelSetOutportBit(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdoLevelSetOutportBit(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdoLevelSetOutportBit(lModuleNo : LongInt; lOffset :
LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in bit unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0 ~ 31	0(LOW), 1(HIGH)
SIO-DB32P	0 ~ 15	0(LOW), 1(HIGH)

SIO-DO32T	0 ~ 31	0(LOW), 1(HIGH)
SIO-DB32T	0 ~ 15	0(LOW), 1(HIGH)

Example

C++

```
// Set data level in bit unit at Offset in module 0.
AxdoLevelSetOutportBit(0, 0, 1);
```

Visual Basic

```
' Set data level in bit unit at Offset in module 0.
AxdoLevelSetOutportBit 0, 0, 1
```

Delphi

```
{ Set data level in bit unit at Offset in module 0. }
AxdoLevelSetOutportBit(0, 0, 1);
```

See Also

AxdoLevelSetOutportByte, AxdoLevelSetOutportWord, AxdoLevelSetOutportDword, AxdoLevelSetOutport

AxdoLevelSetOutportByte

Purpose

Set data level in byte unit at Offset location of the designated output contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelSetOutportByte(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdoLevelSetOutportByte(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdoLevelSetOutportByte(lModuleNo : LongInt; lOffset :
LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uLevel	Level value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in byte unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0, 1, 2, 3	00h ~ FFh
SIO-DB32P	0, 1	00h ~ FFh

SIO-DO32T	0, 1, 2, 3	00h ~ FFh
SIO-DB32T	0, 1	00h ~ FFh

Example

C++

```
// Set data level in byte unit at Offset in module 0.
AxdoLevelSetOutportByte(0, 0, 0xFF);
```

Visual Basic

```
' Set data level in byte unit at Offset in module 0.
AxdoLevelSetOutportByte 0, 0, &hFF
```

Delphi

```
{ Set data level in byte unit at Offset in module 0. }
AxdoLevelSetOutportByte(0, 0, $FF);
```

See Also

AxdoLevelSetOutportBit, AxdoLevelSetOutportWord, AxdoLevelSetOutportDword, AxdoLevelSetOutport

AxdoLevelSetOutportWord

Purpose

Set data level in word unit at Offset location of the designated output contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelSetOutportWord(long lModuleNo, long lOffset, DWORD
uLevel);
```

Visual Basic

```
Function AxdoLevelSetOutportWord(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdoLevelSetOutportWord(lModuleNo : LongInt; lOffset :
LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uLevel	Level value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0, 1	0000h ~ FFFFh
SIO-DB32P	0	0000h ~ FFFFh

SIO-DO32T	0, 1	0000h ~ FFFFh
SIO-DB32T	0	0990h ~ FFFFh

Example

C++

```
// Set data level in word unit at Offset in module 0.
AxdoLevelSetOutportWord(0, 0, 0xFFFF);
```

Visual Basic

```
' Set data level in word unit at Offset in module 0.
AxdoLevelSetOutportWord 0, 0, &hFFFF
```

Delphi

```
{ Set data level in word unit at Offset in module 0. }
AxdoLevelSetOutportWord(0, 0, $FFFF);
```

See Also

AxdoLevelSetOutportBit, AxdoLevelSetOutportByte, AxdoLevelSetOutportDword, AxdoLevelSetOutport

AxdoLevelSetOutportDword

Purpose

Set data level in double word unit at Offset location of the designated output contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelSetOutportDword(long lModuleNo, long lOffset, DWORD uLevel);
```

Visual Basic

```
Function AxdoLevelSetOutportDword(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uLevel As Long) As Long
```

Delphi

```
function AxdoLevelSetOutportDword(lModuleNo : LongInt; lOffset : LongInt; uLevel : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uLevel	Level value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in double word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0	00000000h ~ FFFFFFFFh
SIO-DB32P	0	00000000h ~ FFFFFFFFh

SIO-DO32T	0	00000000h ~ FFFFFFFFh
SIO-DB32T	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Set data level in double word unit at Offset in module 0.
AxdoLevelSetOutportDword(0, 0, 0xFFFFFFFF);
```

Visual Basic

```
' Set data level in double word unit at Offset in module 0.
AxdoLevelSetOutportDword 0, 0, &hFFFFFFFF
```

Delphi

```
{ Set data level in double word unit at Offset in module 0. }
AxdoLevelSetOutportDword(0, 0, $FFFFFFFF);
```

See Also

AxdoLevelSetOutportBit, AxdoLevelSetOutportByte, AxdoLevelSetOutportWord, AxdoLevelSetOutport

AxdoLevelGetOutportBit

Purpose

Check data level in bit unit at Offset location of the designated output contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelGetOutportBit(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdoLevelGetOutportBit(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdoLevelGetOutportBit(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in bit unit at Offset location of the designated output contact module.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0 ~ 31	0(LOW), 1(HIGH)
SIO-DB32P	0 ~ 15	0(LOW), 1(HIGH)

SIO-DO32T	0 ~ 31	0(LOW), 1(HIGH)
SIO-DB32T	0 ~ 15	0(LOW), 1(HIGH)

Example

C++

```
// Check data level in bit unit at Offset in module 0.
DWORD dwLevel;

AxdoLevelGetOutportBit(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in bit unit at Offset in module 0.
Dim lLevel As long

AxdoLevelGetOutportBit 0, 0, lLevel
```

Delphi

```
{ Check data level in bit unit at Offset in module 0. }
var
  dwLevel : DWord;

begin
  AxdoLevelGetOutportBit(0, 0, @dwLevel);
end;
```

See Also

AxdoLevelGetOutportByte, AxdoLevelGetOutportWord, AxdoLevelGetOutportDword, AxdoLevelGetOutport

AxdoLevelGetOutportByte

Purpose

Check data level in byte unit at Offset location of the designated output contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelGetOutportByte(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdoLevelGetOutportByte(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdoLevelGetOutportByte(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upLevel	Level value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in byte unit at Offset location of the output contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0, 1, 2, 3	00h ~ FFh
SIO-DB32P	0, 1	00h ~ FFh

SIO-DO32T	0, 1, 2, 3	00h ~ FFh
SIO-DB32T	0, 1	00h ~ FFh

Example

C++

```
// Check data level in byte unit at Offset in module 0.
DWORD dwLevel;

AxdoLevelGetOutportByte(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in byte unit at Offset in module 0.
Dim lLevel As long

AxdoLevelGetOutportByte 0, 0, lLevel
```

Delphi

```
{ Check data level in byte unit at Offset in module 0. }
var
  dwLevel : DWord;

begin
  AxdoLevelGetOutportByte(0, 0, @dwLevel);
end;
```

See Also

AxdoLevelGetOutportBit, AxdoLevelGetOutportWord, AxdoLevelGetOutportDword, AxdoLevelGetOutport

AxdoLevelGetOutportWord

Purpose

Check data level in word unit at Offset location of the designated output contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelGetOutportWord(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdoLevelGetOutportWord(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdoLevelGetOutportWord(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upLevel	Level value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0, 1	0000h ~ FFFFh
SIO-DB32P	0	0000h ~ FFFFh

SIO-DO32T	0, 1	0000h ~ FFFFh
SIO-DB32T	0	0990h ~ FFFFh

Example

C++

```
// Check data level in word unit at Offset 0 in module 0.
DWORD dwLevel;

AxdoLevelGetOutportWord(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in word unit at Offset 0 in module 0.
Dim lLevel As long

AxdoLevelGetOutportWord 0, 0, lLevel
```

Delphi

```
{ Check data level in word unit at Offset 0 in module 0. }
var
  dwLevel : DWord;

begin
  AxdoLevelGetOutportWord(0, 0, @dwLevel);
end;
```

See Also

AxdoLevelGetOutportBit, AxdoLevelGetOutportByte, AxdoLevelGetOutportDword, AxdoLevelGetOutport

AxdoLevelGetOutportDword

Purpose

Check data level in double word unit at Offset location of the designated output contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelGetOutportDword(long lModuleNo, long lOffset, DWORD
*upLevel);
```

Visual Basic

```
Function AxdoLevelGetOutportDword(ByVal lModuleNo As Long, ByVal
lOffset As Long, ByRef upLevel As Long) As Long
```

Delphi

```
function AxdoLevelGetOutportDword(lModuleNo : LongInt; lOffset :
LongInt; upLevel : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upLevel	Level value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in double word unit at Offset location of the output contact module that user designates

Check which module is the designated module and check the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Level
SIO-DI32	-	-
SIO-DO32P	0	00000000h ~ FFFFFFFFh
SIO-DB32P	0	00000000h ~ FFFFFFFFh

SIO-DO32T	0	00000000h ~ FFFFFFFFh
SIO-DB32T	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Check data level in double word unit at Offset 0 in module 0.
DWORD dwLevel;

AxdoLevelGetOutportDword(0, 0, &dwLevel);
```

Visual Basic

```
' Check data level in double word unit at Offset 0 in module 0.
Dim lLevel As long

AxdoLevelGetOutportDword 0, 0, lLevel
```

Delphi

```
{ Check data level in double word unit at Offset 0 in module 0. }
var
  dwLevel : DWord;

begin
  AxdoLevelGetOutportDword(0, 0, @dwLevel);
end;
```

See Also

AxdoLevelGetOutportBit, AxdoLevelGetOutportByte, AxdoLevelGetOutportWord, AxdoLevelGetOutport

AxdoLevelSetOutport

Purpose

Set data level in bit unit at Offset locations of the entire output contact module.

Setting data level sets whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelSetOutport(long lOffset, DWORD uLevel);
```

Visual Basic

```
Function AxdoLevelSetOutport(ByVal lOffset As Long, ByVal uLevel As
Long) As Long
```

Delphi

```
function AxdoLevelSetOutport(lOffset : LongInt; uLevel : DWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from output contact
[>>]uLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Set data level in bit unit at Offset locations of the entire output contact module.

Offset ranges from 0 to the number of entire input contact-1.

Example

C++

```
// Check data level in double word unit at Offset 0 in the entire module.
AxdoLevelSetOutport(0, 1);
```

Visual Basic

```
' Check data level in double word unit at Offset 0 in the entire module.
AxdoLevelSetOutport 0, 1
```

Delphi

```
{ Check data level in double word unit at Offset 0 in the entire module. }
AxdoLevelSetOutport(0, 1);
```

See Also

AxdoLevelSetOutportBit, AxdoLevelSetOutportByte, AxdoLevelSetOutportWord, AxdoLevelSetOutportDword, AxdoLevelGetOutport

AxdoLevelGetOutport

Purpose

Check data level in bit unit at Offset locations of the entire output contact module.

Checking data level checks whether Active state of signal is HIGH(1) or LOW(0).

Format

C++

```
DWORD AxdoLevelGetOutport(long lOffset, DWORD *upLevel);
```

Visual Basic

```
Function AxdoLevelGetOutport(ByVal lOffset As Long, ByRef upLevel As
Long) As Long
```

Delphi

```
function AxdoLevelGetOutport(lOffset : LongInt; upLevel : PDWord) :
DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from entire output contact
[<<]upLevel	Level value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check data level in bit unit at Offset locations of the entire output contact module.

Offset ranges from 0 to the number of entire input contact-1.

Example

C++

```
// Check data level in bit unit at Offset 0 in the entire module.
DWORD dwLevel;

AxdoLevelGetOutport(0, &dwLevel);
```

Visual Basic

```
' Check data level in bit unit at Offset 0 in the entire module.
Dim lLevel As long

AxdoLevelGetOutport 0, lLevel
```

Delphi

```
{ Check data level in bit unit at Offset 0 in the entire module. }
var
   dwLevel : DWord;

begin
   AxdoLevelGetOutport(0, @dwLevel);
end
```

See Also

AxdoLevelGetOutportBit, AxdoLevelGetOutportByte, AxdoLevelGetOutportWord, AxdoLevelGetOutportDword, AxdoLevelSetOutport

Reading input port

Function	Description
AxdiReadInportBit	Read data in bit unit at Offset location of the designated input contact module.
AxdiReadInportByte	Read data in byte unit at Offset location of the designated input contact module.
AxdiReadInportWord	Read data in word unit at Offset location of the designated input contact module.
AxdiReadInportDword	Read data in double word unit at Offset location of the designated input contact module.
AxdiReadInport	Read data in bit unit at Offset locations of the entire input contact module.

AxdiReadInportBit

Purpose

Format Read data in bit unit at Offset location of the designated input contact module.

Format

C++

```
DWORD AxdiReadInportBit(long lModuleNo, long lOffset, DWORD *upValue);
```

Visual Basic

```
Function AxdiReadInportBit(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiReadInportBit(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upValue	Input contact value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in bit unit at Offset location of the input contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Return value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(Off), 1(On)
SIO-DO32x	-	-
SIO-DB32x	0 ~ 15	0(Off), 1(On)

Example

C++

```
// Read data in bit unit at Offset 0 in module 0.
DWORD dwValue;

AxdiReadInportBit(0, 0, &dwValue);
```

Visual Basic

```
' Read data in bit unit at Offset 0 in module 0.
Dim lValue As Long

AxdiReadInportBit 0, 0, lValue
```

Delphi

```
{ Read data in bit unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdiReadInportBit(0, 0, @dwValue);
end;
```

See Also

AxdiReadInportByte, AxdiReadInportWord, AxdiReadInportDword, AxdiReadInport

AxdiReadInportByte

Purpose

Read data in byte unit at Offset location of the designated input contact module.

Format

C++

```
DWORD AxdiReadInportByte(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdiReadInportByte(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiReadInportByte(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upValue	Input contact value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in byte unit at Offset location of the input contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Return Value

Module	Offset	Value
SIO-DI32	0, 1, 2, 3	00h ~ FFh
SIO-DO32x	-	-
SIO-DB32x	0, 1	00h ~ FFh

Example

C++

```
// Read data in byte unit at Offset 0 in module 0.
DWORD dwValue;

AxdiReadInportByte(0, 0, &dwValue);
```

Visual Basic

```
' Read data in byte unit at Offset 0 in module 0.
Dim lValue As Long

AxdiReadInportByte 0, 0, lValue
```

Delphi

```
{ Read data in byte unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdiReadInportByte(0, 0, @dwValue);
end;
```

See Also

AxdiReadInportBit, AxdiReadInportWord, AxdiReadInportDword, AxdiReadInport

AxdiReadInportWord

Purpose

Read data in word unit at Offset location of the designated input contact module.

Format

C++

```
DWORD AxdiReadInportWord(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdiReadInportWord(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiReadInportWord(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upValue	Input contact value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in word unit at Offset location of the designated input contact module.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range Return value

Module	Offset	Value
SIO-DI32	0, 1	0000h ~ FFFFh
SIO-DO32x	-	-
SIO-DB32x	0	0000h ~ FFFFh

Example

C++

```
// Read data in word unit at Offset 0 in module 0.
DWORD dwValue;

AxdiReadInportWord (0, 0, &dwValue);
```

Visual Basic

```
' Read data in word unit at Offset 0 in module 0.
Dim lValue As Long

AxdiReadInportWord 0, 0, lValue
```

Delphi

```
{ Read data in word unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdiReadInportWord (0, 0, @dwValue);
end;
```

See Also

AxdiReadInportBit, AxdiReadInportByte, AxdiReadInportDword, AxdiReadInport

AxdiReadInportDword

Purpose

Read data in double word unit at Offset location of the designated input contact module.

Format

C++

```
DWORD AxdiReadInportDword(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdiReadInportDword(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiReadInportDword(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upValue	Input contact value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in double word unit at Offset location of the input contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range Return Value

Module	Offset	Value
SIO-DI32	0	00000000h ~ FFFFFFFFh
SIO-DO32x	-	-
SIO-DB32x	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Read data in double word unit at Offset 0 in module 0.
DWORD dwValue;

AxdiReadInportDword (0, 0, &dwValue);
```

Visual Basic

```
' Read data in double word unit at Offset 0 in module 0.
Dim lValue As Long

AxdiReadInportDword 0, 0, lValue
```

Delphi

```
{ Read data in double word unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdiReadInportDword (0, 0, @dwValue);
end;
```

See Also

AxdiReadInportBit, AxdiReadInportByte, AxdiReadInportWord, AxdiReadInport

AxdiReadInport

Purpose

Read data in bit unit at Offset locations of the entire input contact module.

Format

C++

```
DWORD AxdiReadInport(long lOffset, DWORD *upValue);
```

Visual Basic

```
Function AxdiReadInport(ByVal lOffset As Long, ByRef upValue As Long)
As Long
```

Delphi

```
function AxdiReadInport(lOffset : LongInt; upValue : PDWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from entire input contact
[<<]upValue	Input contact value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in bit unit at Offset locations of the entire input contact module.

Offset ranges from 0 to the number of entire input contact-1.

Example

C++

```
// Read data in bit unit at Offset 0 in the entire output module.
DWORD dwValue;

AxdiReadInport(0, &dwValue);
```

Visual Basic

```
' Read data in bit unit at Offset 0 in the entire output module.
Dim lValue As Long

AxdiReadInport 0, lValue
```

Delphi

```
{ Read data in bit unit at Offset 0 in the entire output module. }
var
  dwValue : DWord;

begin
  AxdiReadInport(0, @dwValue);
end;
```

See Also

AxdiReadInportBit, AxdiReadInportByte, AxdiReadInportWord, AxdiReadInportDword

Reading and Writing output port

Function	Description
AxdoWriteOutportBit	Output data in bit unit at Offset location of the designated output contact module.
AxdoWriteOutportByte	Output data in byte unit at Offset location of the designated output contact module.
AxdoWriteOutportWord	Output data in word unit at Offset location of the designated output contact module.
AxdoWriteOutportDword	Output data in double word unit at Offset location of the designated output contact module.
AxdoReadOutportBit	Read data in bit unit at Offset location of the designated output contact module.
AxdoReadOutportByte	Read data in byte unit at Offset location of the designated output contact module.
AxdoReadOutportWord	Read data in word unit at Offset location of the designated output contact module.
AxdoReadOutportDword	Read data in double word unit at Offset location of the designated output contact module.
AxdoWriteOutport	Output data in bit unit at Offset locations of the entire output contact module.
AxdoReadOutport	Read data in bit unit at Offset locations of the entire output contact module.

AxdoWriteOutportBit

Purpose

Output data in bit unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoWriteOutportBit(long lModuleNo, long lOffset, DWORD uValue);
```

Visual Basic

```
Function AxdoWriteOutportBit(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdoWriteOutportBit(lModuleNo : LongInt; lOffset : LongInt;
uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uValue	Output contact value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Output data in bit unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	bValue
SIO-DO32x	0 ~ 31	0(Off), 1(On)
SIO-DB32x	0 ~ 15	0(Off), 1(On)

Example

C++

```
// Output data in bit unit at Offset 0 in output contact module 0.
AxdoWriteOutportBit(0, 0, 1);
```

Visual Basic

```
' Output data in bit unit at Offset 0 in output contact module 0.
AxdoWriteOutportBit 0, 0, 1
```

Delphi

```
{ Output data in bit unit at Offset 0 in output contact module 0. }
AxdoWriteOutportBit(0, 0, 1);
```

See Also

AxdoWriteOutportByte, AxdoWriteOutportWord, AxdoWriteOutportDword, AxdoReadOutportBit, AxdoWriteOutport

AxdoWriteOutportByte

Purpose

Output data in byte unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoWriteOutportByte(long lModuleNo, long lOffset, DWORD uValue);
```

Visual Basic

```
Function AxdoWriteOutportByte(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdoWriteOutportByte(lModuleNo : LongInt; lOffset : LongInt; uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uValue	Output contact value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Output data in byte unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	bValue
SIO-DO32x	0, 1, 2, 3	00h ~ FFh
SIO-DB32x	0, 1	00h ~ FFh

Example

C++

```
// Output data in byte unit at Offset 0 of output contact module 0.
AxdoWriteOutportByte(0, 0, 0xFF);
```

Visual Basic

```
' Output data in byte unit at Offset 0 of output contact module 0.
AxdoWriteOutportByte 0, 0, &hFF
```

Delphi

```
{ Output data in byte unit at Offset 0 of output contact module 0. }
AxdoWriteOutportByte (0, 0, $FF)
```

See Also

AxdoWriteOutportBit, AxdoWriteOutportWord, AxdoWriteOutportDword, AxdoReadOutportByte, AxdoWriteOutport

AxdoWriteOutportWord

Purpose

Output data in word unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoWriteOutportByte(long lModuleNo, long lOffset, DWORD
uValue);
```

Visual Basic

```
Function AxdoWriteOutportByte(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdoWriteOutportByte(lModuleNo : LongInt; lOffset : LongInt;
uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uValue	Output contact value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Output data in word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	bValue
SIO-DO32x	0,1	0000h ~ FFFFh
SIO-DB32x	0	0000h ~ FFFFh

Example

C++

```
// Output data in word unit at Offset 0 of output contact module 0.
AxdoWriteOutportByte(0, 0, 0xFFFF);
```

Visual Basic

```
' Output data in word unit at Offset 0 of output contact module 0.
AxdoWriteOutportByte 0, 0, &hFFFF
```

Delphi

```
{ Output data in word unit at Offset 0 of output contact module 0. }
AxdoWriteOutportByte(0, 0, $FFFF);
```

See Also

AxdoWriteOutportBit, AxdoWriteOutportByte, AxdoWriteOutportDword, AxdoReadOutportWord, AxdoWriteOutport

AxdoWriteOutportDword

Purpose

Output data in double word unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoWriteOutportDword(long lModuleNo, long lOffset, DWORD uValue);
```

Visual Basic

```
Function AxdoWriteOutportDword(ByVal lModuleNo As Long, ByVal lOffset As Long, ByVal uValue As Long) As Long
```

Delphi

```
function AxdoWriteOutportDword(lModuleNo : LongInt; lOffset : LongInt; uValue : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[>>]uValue	Output contact value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Output data in double word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and set the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	bValue
SIO-DO32x	0	00000000h ~ FFFFFFFFh
SIO-DB32x	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Output data in double word unit at Offset 0 in output contact module 0.
AxdoWriteOutportDword(0, 0, 0xFFFFFFFF);
```

Visual Basic

```
' Output data in double word unit at Offset 0 in output contact module 0.
AxdoWriteOutportDword 0, 0, &hFFFFFFFF
```

Delphi

```
{ Output data in double word unit at Offset 0 in output contact module 0. }
AxdoWriteOutportDword(0, 0, $FFFFFFFF);
```

See Also

AxdoWriteOutportBit, AxdoWriteOutportByte, AxdoWriteOutportWord, AxdoReadOutportDword, AxdoWriteOutport

AxdoReadOutportBit

Purpose

Read data in bit unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoReadOutportBit(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdoReadOutportBit(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdoReadOutportBit(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upValue	Output contact value (Bit)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in bit unit at Offset location of the output contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	-	-
SIO-DO32x	0 ~ 31	0(Off), 1(On)
SIO-DB32x	0 ~ 15	0(Off), 1(On)

Example

C++

```
// Read data in bit unit at Offset 0 in module 0.
DWORD dwValue;

AxdoReadOutportBit(0, 0, &dwValue);
```

Visual Basic

```
' Read data in bit unit at Offset 0 in module 0.
Dim lValue As Long

AxdoReadOutportBit 0, 0, lValue
```

Delphi

```
{ Read data in bit unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdoReadOutportBit(0, 0, @dwValue);
end;
```

See Also

AxdoReadOutportByte, AxdoReadOutportWord, AxdoReadOutportDword, AxdoWriteOutportBit, AxdoReadOutport

AxdoReadOutportByte

Purpose

Read data in byte unit at Offset location of the designated output contact module.

Format

C++

```
Function AxdoReadOutportByte(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdoReadOutportByte(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdoReadOutportByte(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upValue	Output contact value (Byte)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in byte unit at Offset location of the output contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	-	-
SIO-DO32x	0, 1, 2, 3	00h ~ FFh
SIO-DB32x	0, 1	00h ~ FFh

Example

C++

```
// Read data in byte unit at Offset 0 in module 0.
DWORD dwValue;

AxdoReadOutportByte(0, 0, &dwValue);
```

Visual Basic

```
' Read data in byte unit at Offset 0 in module 0.
Dim lValue As Long

AxdoReadOutportByte 0, 0, lValue
```

Delphi

```
{ Read data in byte unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdoReadOutportByte(0, 0, @dwValue);
end;
```

See Also

AxdoReadOutportBit, AxdoReadOutportWord, AxdoReadOutportDword, AxdoWriteOutportByte, AxdoReadOutport

AxdoReadOutportWord

Purpose

Read data in word unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoReadOutportWord(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdoReadOutportWord(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdoReadOutportWord(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upValue	Output contact value (Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	-	-
SIO-DO32x	0, 1	0000h ~ FFFFh
SIO-DB32x	0	0000h ~ FFFFh

Example

C++

```
// Read data in word unit at Offset 0 in module 0.
DWORD dwValue;

AxdoReadOutportWord(0, 0, &dwValue);
```

Visual Basic

```
' Read data in word unit at Offset 0 in module 0.
Dim lValue As Long

AxdoReadOutportWord 0, 0, lValue
```

Delphi

```
{ Read data in word unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdoReadOutportWord(0, 0, @dwValue);
end;
```

See Also

AxdoReadOutportBit, AxdoReadOutportByte, AxdoReadOutportDword, AxdoWriteOutportWord, AxdoReadOutport

AxdoReadOutportDword

Purpose

Read data in double word unit at Offset location of the designated output contact module.

Format

C++

```
DWORD AxdoReadOutportDword(long lModuleNo, long lOffset, DWORD
*upValue);
```

Visual Basic

```
Function AxdoReadOutportDword(ByVal lModuleNo As Long, ByVal lOffset
As Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdoReadOutportDword(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from output contact
[<<]upValue	Output contact value (Double Word)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in double word unit at Offset location of the output contact module that user designates.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	-	-
SIO-DO32x	0	00000000h ~ FFFFFFFFh
SIO-DB32x	0	00000000h ~ FFFFFFFFh

Example

C++

```
// Read data in double word unit at Offset 0 in module 0.
DWORD dwValue;

AxdoReadOutportDword(0, 0, &dwValue);
```

Visual Basic

```
' Read data in double word unit at Offset 0 in module 0.
Dim lValue As Long

AxdoReadOutportDword 0, 0, lValue
```

Delphi

```
{ Read data in double word unit at Offset 0 in module 0. }
var
  dwValue : DWord;

begin
  AxdoReadOutportDword(0, 0, @dwValue);
end;
```

See Also

AxdoReadOutportBit, AxdoReadOutportByte, AxdoReadOutportWord, AxdoWriteOutportDword, AxdoReadOutport

AxdoWriteOutport

Purpose

Output data in bit unit at Offset locations of the entire output contact module.

Format

C++

```
DWORD AxdoWriteOutport(long lOffset, DWORD uValue);
```

Visual Basic

```
Function AxdoWriteOutport(ByVal lOffset As Long, ByVal uValue As Long)
As Long
```

Delphi

```
Function AxdoWriteOutport(lOffset : LongInt; uValue : DWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from entire output contact
[>>]uValue	Output contact value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Output data in bit unit at Offset locations of the entire output contact module.

Offset ranges from 0 to the number of entire output contact-1.

Example

C++

```
// Output data in bit unit at Offset 0 in the entire output contact module.
AxdoWriteOutport(0, 1);
```

Visual Basic

```
' Output data in bit unit at Offset 0 in the entire output contact module.
AxdoWriteOutport 0, 1
```

Delphi

```
{ Output data in bit unit at Offset 0 in the entire output contact module. }
AxdoWriteOutport(0, 1);
```

See Also

AxdoWriteOutportBit, AxdoWriteOutportByte, AxdoWriteOutportWord, AxdoWriteOutportDword, AxdoReadOutport

AxdoReadOutport

Purpose

Read data in bit unit at Offset locations of the entire output contact module.

Format

C++

```
DWORD AxdoReadOutport(long lOffset, DWORD *upValue);
```

Visual Basic

```
Function AxdoReadOutport(ByVal lOffset As Long, ByRef upValue As Long)
As Long
```

Delphi

```
function AxdoReadOutport(lOffset : LongInt; upValue : PDWord) : DWord;
stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lOffset	Offset location from entire output contact
[<<]upValue	Output contact value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Read data in bit unit at Offset locations of the output contact module that user designates.

Offset ranges from 0 to the number of entire output contact-1.

Example

C++

```
// Read data in bit unit at Offset 0 in the entire output module.
DWORD dwValue;

AxdoReadOutport(0, &dwValue);
```

Visual Basic

```
' Read data in bit unit at Offset 0 in the entire output module.
Dim lValue As Long

AxdoReadOutport 0, lValue
```

Delphi

```
{ Read data in bit unit at Offset 0 in the entire output module. }
var
   dwValue : DWord;

begin
   AxdoReadOutport(0, @dwValue);
end;
```

See Also

AxdoReadOutportBit, AxdoReadOutportByte, AxdoReadOutportWord, AxdoReadOutportDword, AxdoWriteOutport

Advanced APIs

Function	Description
AxdiIsPulseOn	Check if signal at Offset location of the designated input contact module is turned On from Off.
AxdiIsPulseOff	Check if signal at Offset location of the designated input contact module is turned Off from On
AxdiIsOn	Check if signal at Offset location of the designated input contact module keeps On until it is called up to count times.
AxdiIsOff	Check if signal at Offset location of the designated input contact module keeps Off until it is called up to count times.
AxdoOutPulseOn	Keep signal On at Offset location of the designated output contact module for the set mSec and turn it Off after the time.
AxdoOutPulseOff	Keep signal Off at Offset location of the designated output contact module for the set mSec and turn it On after the time.
AxdoToggleStart	Keep signal Off at Offset location of the designated output contact module for the set mSec and turn it On after the time.
AxdoToggleStop	Stop output toggling at Offset location of the designated output contact module in the set signal state.

AxdiIsPulseOn

Purpose

Check if signal at Offset location of the designated input contact module is turned On from Off.

Format

C++

```
DWORD AxdiIsPulseOn(long lModuleNo, long lOffset, DWORD *upValue);
```

Visual Basic

```
Function AxdiIsPulseOn(ByVal lModuleNo As Long, ByVal lOffset As Long,
ByRef upValue As Long) As Long
```

Delphi

```
function AxdiIsPulseOn(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upValue	Whether the state is On (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check if signal at Offset location of the input contact module that user designates is turned On from Off.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(False), 1(True)
SIO-DB32x	0 ~ 15	0(False), 1(True)

Example

C++

```
// Check if signal at Offset location of module 0 is turned to On from Off.
DWORD dwValue;

AxdiIsPulseOn(0, 0, &dwValue);
```

Visual Basic

```
' Check if signal at Offset location of module 0 is turned to On from Off.
Dim lValue As Long

AxdiIsPulseOn 0, 0, lValue
```

Delphi

```
{ Check if signal at Offset location of module 0 is turned to On from Off. }
var
  dwValue : DWord;

begin
  AxdiIsPulseOn(0, 0, @dwValue);
end;
```

See Also

AxdiIsPulseOff, AxdiIsOn, AxdiIsOff, AxdoOutPulseOn, AxdoOutPulseOff, AxdoToggleStart, AxdoToggleStop

AxdiIsPulseOff

Purpose

Check if signal at Offset location of the designated input contact module is turned Off from On.

Format

C++

```
DWORD AxdiIsPulseOff(long lModuleNo, long lOffset, DWORD *upValue);
```

Visual Basic

```
Function AxdiIsPulseOff(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByRef upValue As Long) As Long
```

Delphi

```
function AxdiIsPulseOff(lModuleNo : LongInt; lOffset : LongInt;
upValue : PDWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[<<]upValue	Whether the state is Off (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check if signal at Offset location of the input contact module that user designates is turned Off from On.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(False), 1(True)
SIO-DB32x	0 ~ 15	0(False), 1(True)

Example

C++

```
// Check if signal at Offset location of module 0 is turned Off from On.
DWORD dwValue;

AxdiIsPulseOff(0, 0, &dwValue);
```

Visual Basic

```
' Check if signal at Offset location of module 0 is turned Off from On.
Dim lValue As Long

AxdiIsPulseOff 0, 0, lValue
```

Delphi

```
{ Check if signal at Offset location of module 0 is turned Off from On. }
var
  dwValue : DWord;

begin
  AxdiIsPulseOff(0, 0, @dwValue);
end;
```

See Also

AxdiIsPulseOn, AxdiIsOn, AxdiIsOff, AxdoOutPulseOn, AxdoOutPulseOff, AxdoToggleStart, AxdoToggleStop

AxdiIsOn

Purpose

Check if signal at Offset location of the designated input contact module keeps On until it is called up to count times.

Format

C++

```
DWORD AxdiIsOn(long lModuleNo, long lOffset, long lCount, DWORD
*upValue, long lStart);
```

Visual Basic

```
Function AxdiIsOn(ByVal lModuleNo As Long, ByVal lOffset As Long,
ByVal lCount As Long, ByRef upValue As Long, ByVal lStart As Long) As
Long
```

Delphi

```
function AxdiIsOn(lModuleNo : LongInt; lOffset : LongInt; lCount :
LongInt; upValue : PDWord; lStart : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]lCount	The number of calling up
[>>]lStart	Start checking signal keeping (Initial call-up: 1, Follow-up calls: 0)
[<<]upValue	Whether the state is kept On. (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check if signal at Offset location of the input contact module that user designates keeps On until it is called up to count number.

Set lStart as 1 in the initial call-up. Then, during the follow-up calls up to count-1 times, set it as 0. The return value for upValue must be FALSE for the follow-up calls up to count-1 times. From the count th call-up, return TRUE if signal has kept On state during previous call-ups. If not, return FALSE on. In other words, the return value from the countth call-up is the value that determines the state of signal continuity. For example, if lCount is set as 10 and signal is kept On, the return value will be

TRUE as signal is kept On unless either the initialization of count to 0 after calling up 10 times or restarting it by setting lStart as 1 takes place.

If count, which is the call-up number, is set as 0, the number can be used to check whether signal is On. And it also can be used to initialize the call-up number which has been set before.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(FALSE), 1(TRUE)
SIO-DB32x	0 ~ 15	0(FALSE), 1(TRUE)

Example

C++

```
// Check if signal at Offset 0 of module 0 keeps On until it is called up
// 10 (Count) times.
// Example 1.
   short i;
   DWORD dwValue;
   AxdiIsOn(0, 0, 10, &dwValue, 1);
   // Initial call-up, Start checking the state of signal continuity during 10
   // call-ups lStart:1
   for (i = 0; i < 9; i++)
   // Check if signal keeps On during 10 call-ups including initial call-up.
   {
      // In the actual program, call-up routine that has constant period through
      // either Timer or Thread is used.
      AxdiIsOn(0, 0, 10, &dwValue, 0);
      // Make follow-up calls up to count times afterwards.
   }

   if (dwValue != 0)
      AfxMessageBox ("Signal has kept ON.");

// Exmaple 2. (The codes below are the same as above, yet clearer.)
   short i;
   DWORD dwValue;
   AxdiIsOn(0, 0, 0, &dwValue, 1);
   // As lCount is 0, previous Count is initialized.
   If (dwValue !=0)        // The state of current signal is On.
   {
      for (i = 0; i < 10; i++)
      // Check if signal keeps On during 10 call-ups including initial call-up.
      {
         // In actual program, call-up routine that has constant period through
```

```
        // either Timer or Thread is used.
        AxdiIsOn(0, 0, 10, &dwValue, 0);
        // Make follow-up calls with new count applied.
     }
  if (dwValue != 0)
     AfxMessageBox ("Signal has kept ON.");
}
```

Visual Basic

```
' Check if signal at Offset 0 of module 0 keeps On until it is called up
' 10 (Count) times.
Dim i As Integer
Dim lValue As Long

AxdiIsOn 0, 0, 10, lValue, 1
' Initial call-up, Start checking the state of signal continuity during 10 call-ups.

For I = 0 To 9
  AxdiIsOn 0, 0, 10, lValue, 0
Next

If lValue = 1 Then
  MsgBox "Signal has kept On."
End If
```

Delphi

```
{ Check if signal at Offset 0 of module 0 keeps On until it is called up 10 times. }
var
  i : SmallInt;
  dwValue : DWord;

begin
  begin
     AxdiIsOn(0, 0, 10, @dwValue, 1);
  end
  for i := 0 to 9 do
  begin
     AxdiIsOn(0, 0, 10, @dwValue, 0);
  end;

  if(dwValue = ON) Then
     Application.MessageBox('Signal has kept ON.', 'Ajinextek', MB_OK);
end;
```

See Also

AxdiIsPulseOn, AxdiIsPulseOff, AxdiIsOff, AxdoOutPulseOn, AxdoOutPulseOff, AxdoToggleStart, AxdoToggleStop

AxdiIsOff

Purpose

Check if signal at Offset location of the designated input contact module keeps Off until it is called up to count times.

Format

C++

```
DWORD AxdiIsOff(long lModuleNo, long lOffset, long lCount, DWORD
*upValue, long lStart);
```

Visual Basic

```
Function AxdiIsOff(ByVal lModuleNo As Long, ByVal lOffset As Long,
ByVal lCount As Long, ByRef upValue As Long, ByVal lStart As Long) As
Long
```

Delphi

```
function AxdiIsOff(lModuleNo : LongInt; lOffset : LongInt; lCount :
LongInt; upValue : PDWord; lStart : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]lCount	The number of calling up
[>>]lStart	Start checking signal keeping (Initial call-up: 1, Follow-up calls: 0)
[<<]upValue	Whether the state is Off. (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Check if signal at Offset location of the input contact module that user designates keeps Off until it is called up to count times.

Set lStart as 1 in the initial call-up. Then, during the follow-up calls up to count-1 times, set it as 0. The return value for upValue must be FALSE for the follow-up calls up to count-1 times. From the count th call-up, return TRUE if signal has kept Off state during previous call-ups. If not, return FALSE on. That is, the return value from the countth call-up is the value that determines the state of signal continuity. For example, if lCount is set as 10 and signal is kept Off, the return value will be TRUE as

signal is kept Off unless either the initialization of count to 0 after calling up 10 times or restarting it by setting lStart as 1 takes place.

If count, which is the call-up number, is set as 0, the number can be used to check whether signal is On. And it also can be used to initialize the call-up number which has been set.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(FALSE), 1(TRUE)
SIO-DB32x	0 ~ 15	0(FALSE), 1(TRUE)

Example

C++

```
// Check if signal at Offset 0 of module 0 keeps Off until it is called up
// 10 (Count) times.
// Example 1.
   short i;
   DWORD dwValue;
   AxdiIsOff(0,0,0,10,&dwValue,1);
   // Initial call-up, Start checking the state of signal continuity during 10
   // call-ups. lStart:1
   for (i = 0; i < 9; i++)
   // Check if signal keeps Off during 10 call-ups including initial call-up.
   {
      // In actual program, call-up routine that has constant period through
      // either Timer or Thread is used.
      AxdiIsOff(0, 0, 10, &dwValue, 0);
      // Make follow-up calls up to count times afterwards.
   }

   if (dwValue == 1)
      AfxMessageBox ("Signal has kept OFF.");

// Exmaple 2. (The codes below are the same as above, yet clearer.)
   short i;
   DWORD dwValue;
   AxdiIsOff(0, 0, 0, &dwValue, 1);
   // As lCount is 0, previous Count is initialized.
   If (dwValue == 1)    // The state of current signal is Off.
   {
      for (i = 0; i < 10; i++)
      // Check if signal keeps On during 10 call-ups including initial call-up.
      {
         //In actual program, call-up routine that has constant period through
```

```
        // either Timer or Thread is used.
        AxdiIsOff(0, 0, 10, &dwValue, 0);
        // Make follow-up calls with new count applied.
     }
  if (dwValue == 1)
     AfxMessageBox ("Signal has kept OFF.");
}
```

Visual Basic

```
' Check if signal at Offset 0 of module 0 keeps Off until it is called up
' 10 (Count) times.
Dim i As Integer
Dim lValue As Long

AxdiIsOff 0, 0, 10, lValue, 1
' Initial call-up, Start checking the state of signal continuity during 10 call-ups

For I = 0 To 9
  AxdiIsOff 0, 0, 10, lValue, 0
Next

If lValue = 1 Then
  MsgBox "Signal has kept OFF."
End If
```

Delphi

```
{ Check if signal at Offset 0 of module 0 keeps Off until it is called up 10(Count)
times. }
var
  i : SmallInt;
  dwValue : DWord;

begin
  begin
     AxdiIsOff(0, 0, 10, @dwValue, 1);
  end
  for i := 0 to 9 do
  begin
     AxdiIsOff(0, 0, 10, @dwValue, 0);
  end;
  if(dwValue = 1) Then
     Application.MessageBox('Signal has kept OFF.', 'Ajinextek', MB_OK);
end;
```

See Also

AxdiIsPulseOn, AxdiIsPulseOff, AxdiIsOn, AxdoOutPulseOn, AxdoOutPulseOff, AxdoToggleStart, AxdoToggleStop

AxdoOutPulseOn

Purpose

Keep signal on at Offset location of the designated output contact module for the set mSec and turn it off after the time.

Format

C++

```
DWORD AxdoOutPulseOn(long lModuleNo, long lOffset, long lmSec);
```

Visual Basic

```
Function AxdoOutPulseOn(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByVal lmSec As Long) As Long
```

Delphi

```
function AxdoOutPulseOn(lModuleNo : LongInt; lOffset : LongInt;
lmSec : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]lmSec	Keeping time (mSec)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Keep signal On at Offset location of the output contact module that user designates for the set mSec and turn it off after the time.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(FALSE), 1(TRUE)
SIO-DB32x	0 ~ 15	0(FALSE), 1(TRUE)

Example

C++

```
// Keep signal On for 10 seconds and turn it Off at Offset 0 in Module 0.
AxdoOutPulseOn(0, 0, 10000);
```

Visual Basic

```
' Keep signal On for 10 seconds and turn it Off at Offset 0 in Module 0.
AxdoOutPulseOn 0, 0, 10000
```

Delphi

```
{ Keep signal On for 10 seconds and turn it Off at Offset 0 in Module 0. }
AxdoOutPulseOn(0, 0, 10000);
```

See Also

AxdiIsPulseOn, AxdiIsPulseOff, AxdiIsOn, AxdiIsOff, AxdoOutPulseOff, AxdoToggleStart, AxdoToggleStop

AxdoOutPulseOff

Purpose

Keep signal Off at Offset location of the designated output contact module for the set mSec and turn it On after the time.

Format

C++

```
DWORD AxdoOutPulseOff(long lModuleNo, long lOffset, long lmSec);
```

Visual Basic

```
Function AxdoOutPulseOff(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByVal lmSec As Long) As Long
```

Delphi

```
function AxdoOutPulseOff(lModuleNo : LongInt; lOffset : LongInt;
lmSec : LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]lmSec	Keeping time (mSec)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Keep signal Off at Offset location of the output contact module that user designates for the set mSec and turn it On after the time.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(FALSE), 1(TRUE)
SIO-DB32x	0 ~ 15	0(FALSE), 1(TRUE)

Example

C++

```
// Keep signal Off for 10 seconds and turn it On at Offset 0 in Module 0.
AxdoOutPulseOff(0, 0, 10000);
```

Visual Basic

```
' Keep signal Off for 10 seconds and turn it On at Offset 0 in Module 0.
AxdoOutPulseOff 0, 0, 10000
```

Delphi

```
{ Keep signal Off for 10 seconds and turn it On at Offset 0 in Module 0. }
AxdoOutPulseOff(0, 0, 10000);
```

See Also

AxdiIsPulseOn, AxdiIsPulseOff, AxdiIsOn, AxdiIsOff, AxdoOutPulseOn, AxdoToggleStart, AxdoToggleStop

AxdoToggleStart

Purpose

Toggle signal at Offset location of the designated output contact module with the state of initial signal, the set number of repetition and the set On/Off time interval. Keep the initial output state after the toggle.

The set number –1 means infinite repetition.

Format

C++

```
DWORD AxdoToggleStart(long lModuleNo, long lOffset, long lInitState,
long lmSecOn, long lmSecOff, long lCount);
```

Visual Basic

```
Function AxdoToggleStart(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByVal lInitState As Long, ByVal lmSecOn As Long, ByVal lmSecOff
As Long, ByVal lCount As Long) As Long
```

Delphi

```
function AxdoToggleStart(lModuleNo : LongInt; lOffset : LongInt;
lInitState : LongInt; lmSecOn : LongInt;  lmSecOff : LongInt; lCount :
LongInt) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]lInitState	Setting the state of initial signal (Off: 0, On:1)
[>>]lmSecOn	Among toggle intervals, interval time for On Time (mSec)
[>>]lmSecOff	Among toggle intervals, interval time for On Time (mSec)
[>>]lCount	For –1 case, toggle number is infinite.

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Toggle signal at Offset location of the designated output contact module with the state of initial signal, the set number of repetition and the set interval.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(FALSE), 1(TRUE)
SIO-DB32x	0 ~ 15	0(FALSE), 1(TRUE)

Example

C++

```
// Toggle signal in initial Off state for 10 times at the interval of 0.5/1 second at
// Offset 0 in Module 0.
AxdoToggleStart(0, 0, 0, 500, 1000, 10);
```

Visual Basic

```
' Toggle signal in initial Off state for 10 times at the interval of 0.5/1 second at
' Offset 0 in Module 0.
AxdoToggleStart 0, 0, 0, 500, 1000, 10
```

Delphi

```
{ Toggle signal in initial Off state for 10 times at the interval of 0.5/1 second at
Offset 0 in Module 0. }
AxdoToggleStart(0, 0, 0, 500, 1000, 10);
```

See Also

AxdiIsPulseOn, AxdiIsPulseOff, AxdiIsOn, AxdiIsOff, AxdoOutPulseOn, AxdoOutPulseOff, AxdoToggleStop

AxdoToggleStop

Purpose

Stop output toggling at Offset location of the designated output contact module in the set signal state.

Format

C++

```
DWORD AxdoToggleStop(long lModuleNo, long lOffset, DWORD uOnOff);
```

Visual Basic

```
Function AxdoToggleStop(ByVal lModuleNo As Long, ByVal lOffset As
Long, ByVal uOnOff As Long) As Long
```

Delphi

```
function AxdoToggleStop(lModuleNo : LongInt; lOffset : LongInt;
uOnOff : DWord) : DWord; stdcall;
```

Parameters

[in/out] Name	[Init Value] Explanation
[>>]lModuleNo	Module No
[>>]lOffset	Offset location from input contact
[>>]uOnOff	Ouput contact value (Boolean)

Return Values

[0000] AXT_RT_SUCCESS : API function executed successfully
[3051] AXT_RT_DIO_NOT_MODULE : No DIO module
[3101] AXT_RT_DIO_INVALID_MODULE_NO : Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO : Invalid DIO OFFSET number
* See error code Table for more information on status error codes

Description

Stop output toggling at Offset location of the output contact module that user designates in the set signal state.

Check which module is the designated module and read the value referring to Offset range in the table below.

Offset range and Value

Module	Offset	Value
SIO-DI32	0 ~ 31	0(FALSE), 1(TRUE)
SIO-DB32x	0 ~ 15	0(FALSE), 1(TRUE)

Example

C++

```
// Stop toggling and turn signal OFF at Offset 0 in Module 0.
AxdoToggleStop(0, 0, OFF);
```

Visual Basic

```
' Stop toggling and turn signal OFF at Offset 0 in Module 0.
AxdoToggleStop 0, 0, 0t
```

Delphi

```
{ Stop toggling and turn signal OFF at Offset 0 in Module 0. }
AxdoToggleStop(0, 0, OFF)
```

See Also

AxdiIsPulseOn, AxdiIsPulseOff, AxdiIsOn, AxdiIsOff, AxdoOutPulseOn, AxdoOutPulseOff, AxdoToggleStart

Error Code Table Check

Error coded returned from the library functions can be used to check the errors generated during the execution of library functions and to process the result accordingly. Error code returned from each of the library functions indicates reasons of failure of corresponding function, and you can check the error code to understand the cause of failure. In general, it is good idea to check the code when the function returns the code other than AXT_RT_SUCCESS for the purpose of easier debugging.

Error Code	Description
[0000] AXT_RT_SUCCESS	API function executed successfully
[1001] AXT_RT_OPEN_ERROR	Library is not open
[1002] AXT_RT_OPEN_ALREADY	Library is already open
[1053] AXT_RT_NOT_OPEN	Failed to initialize the library
[1054] AXT_RT_NOT_SUPPORT_VERSION	Hardware not supported
[1101] AXT_RT_INVALID_BOARD_NO	Invalid board number
[1102] AXT_RT_INVALID_MODULE_POS	Invalid module position
[1103] AXT_RT_INVALID_LEVEL	Invalid level
[1151] AXT_RT_ERROR_VERSION_READ	Failed to read library version
[1152] AXT_RT_NETWORK_ERROR	Slave nodes is not connected to the network or connection is faulty
[1153] AXT_RT_NETWORK_LOCK_MISMATCH	Stored connection information and the actual connection information is incorrect. Lock of the board in EzConfig information does not match with the actual connection status. Lock information for reconstruction of a slave existing Ezconfig should give clear in the Unlock
[1160] AXT_RT_1ST_BELOW_MIN_VALUE	First parameter is below the minimum value
[1161] AXT_RT_1ST_ABOVE_MAX_VALUE	First parameter is above the maximum value
[1170] AXT_RT_2ND_BELOW_MIN_VALUE	Second parameter is below the minimum value
[1171] AXT_RT_2ND_ABOVE_MAX_VALUE	Second parameter is above the maximum value
[1180] AXT_RT_3RD_BELOW_MIN_VALUE	Third parameter is below the minimum value
[1181] AXT_RT_3RD_ABOVE_MAX_VALUE	Third parameter is above the maximum value
[1190] AXT_RT_4TH_BELOW_MIN_VALUE	Fourth parameter is below the minimum value
[1191] AXT_RT_4TH_ABOVE_MAX_VALUE	Fourth parameter is above the maximum value
[1200] AXT_RT_5TH_BELOW_MIN_VALUE	Fifth parameter is below the minimum value
[1201] AXT_RT_5TH_ABOVE_MAX_VALUE	Fifth parameter is above the maximum value
[1210] AXT_RT_6TH_BELOW_MIN_VALUE	Sixth parameter is below the minimum value
[1211] AXT_RT_6TH_ABOVE_MAX_VALUE	Sixth parameter is above the maximum value
[1220] AXT_RT_7TH_BELOW_MIN_VALUE	Seventh parameter is below the minimum value
[1221] AXT_RT_7TH_ABOVE_MAX_VALUE	Seventh parameter is above the maximum value
[1230] AXT_RT_8TH_BELOW_MIN_VALUE	Eighth parameter is below the minimum value

[1231] AXT_RT_8TH_ABOVE_MAX_VALUE	Eighth parameter is above the maximum value
[1240] AXT_RT_9TH_BELOW_MIN_VALUE	Nineth parameter is below the minimum value
[1241] AXT_RT_9TH_ABOVE_MAX_VALUE	Nineth parameter is above the maximum value
[1250] AXT_RT_10TH_BELOW_MIN_VALUE	Tenth parameter is below the minimum value
[1251] AXT_RT_10TH_ABOVE_MAX_VALUE	Tenth parameter is above the maximum value
[2001] AXT_RT_AIO_OPEN_ERROR	Failed to open AIO module
[2051] AXT_RT_AIO_NOT_MODULE	No AIO module
[2052] AXT_RT_AIO_NOT_EVENT	Failed to read AIO event
[2101] AXT_RT_AIO_INVALID_MODULE_NO	Invalid AIO module
[2102] AXT_RT_AIO_INVALID_CHANNEL_NO	Invalid AIO channel number
[2106] AXT_RT_AIO_INVALID_USE	Can not use AIO
[2107] AXT_RT_AIO_INVALID_TRIGGER_MODE	Invalid trigger mode
[3001] AXT_RT_DIO_OPEN_ERROR	Failed to open DIO module
[3051] AXT_RT_DIO_NOT_MODULE	No DIO module
[3052] AXT_RT_DIO_NOT_INTERRUPT	DIO Interrupt not set
[3101] AXT_RT_DIO_INVALID_MODULE_NO	Invalid DIO module number
[3102] AXT_RT_DIO_INVALID_OFFSET_NO	Invalid DIO OFFSET number
[3103] AXT_RT_DIO_INVALID_LEVEL	Invalid DIO level
[3104] AXT_RT_DIO_INVALID_MODE	Invalid DIO mode
[3105] AXT_RT_DIO_INVALID_VALUE	Invalid setting value
[3106] AXT_RT_DIO_INVALID_USE	Can not use DIO API function
[4001] AXT_RT_MOTION_OPEN_ERROR	Failed to open motion library
[4051] AXT_RT_MOTION_NOT_MODULE	No motion module is installed in the system
[4052] AXT_RT_MOTION_NOT_INTERRUPT	Failed to read the resul of interrupt
[4053] AXT_RT_MOTION_NOT_INITIAL_AXIS_NO	Failed to initialize the motion of corresponding axis
[4054] AXT_RT_MOTION_NOT_IN_CONT_INTERPOL	Command to stop continuous interpolation while not in continuous interpolation motion
[4055] AXT_RT_MOTION_NOT_PARA_READ	Failed to load parameters for home return drive
[4101] AXT_RT_MOTION_INVALID_AXIS_NO	Corresponding axis does not exist
[4102] AXT_RT_MOTION_INVALID_METHOD	Invalid setting for corresponding axis drive
[4103] AXT_RT_MOTION_INVALID_USE	Invalid setting for 'uUse' parameter
[4104] AXT_RT_MOTION_INVALID_LEVEL	Invalid setting for 'uLevel' parameter
[4105] AXT_RT_MOTION_INVALID_BIT_NO	Invalid setting for general purpose input/output bit
[4106] AXT_RT_MOTION_INVALID_STOP_MODE	Invalid setting values for motion stop mode
[4107] AXT_RT_MOTION_INVALID_TRIGGER_MODE	Invalid setting for trigger setting mode
[4108] AXT_RT_MOTION_INVALID_TRIGGER_LEVEL	Invalid setting for trigger output level

[4109] AXT_RT_MOTION_INVALID_SELECTION	'uSelection' parameter is set to a value other than COMMAND or ACTUAL
[4110] AXT_RT_MOTION_INVALID_TIME	Invalid setting for Trigger output time value
[4111] AXT_RT_MOTION_INVALID_FILE_LOAD	Failed to load the file containing motion setting values
[4112] AXT_RT_MOTION_INVALID_FILE_SAVE	Failed to save the file containing motion setting values
[4113] AXT_RT_MOTION_INVALID_VELOCITY	Motion error occurred since the motion drive velocity value is set as zero
[4114] AXT_RT_MOTION_INVALID_ACCELTIME	Motion error occurred since motion drive acceleration time value is set as zero
[4115] AXT_RT_MOTION_INVALID_PULSE_VALUE	Input pulse value is set to a value less than zero when setting motion unit
[4116] AXT_RT_MOTION_INVALID_NODE_NUMBER	Called position or velocity override function while not in motion
[4117] AXT_RT_MOTION_INVALID_TARGET	Flag indicating the cause of multi axis motion stop
[4151] AXT_RT_MOTION_ERROR_IN_NONMOTION	Not in motion drive when it should be
[4152] AXT_RT_MOTION_ERROR_IN_MOTION	Called another motion drive fuction before the completetion of current motion
[4153] AXT_RT_MOTION_ERROR	Error occurred during the execution of multi axis motion stop function
[4154] AXT_RT_MOTION_ERROR_GANTRY_ENABLE	Gantry-enable is activated again while in motion with gantry enabled
[4155] AXT_RT_MOTION_ERROR_GANTRY_AXIS	Invalid input of gantry axis master channel (axis) number (starting from zero)
[4156] AXT_RT_MOTION_ERROR_MASTER_SERVOON	Servo ON is not enabled for the master axis
[4157] AXT_RT_MOTION_ERROR_SLAVE_SERVOON	Servo ON is not enabled for the slavve axis
[4158] AXT_RT_MOTION_INVALID_POSITION	Not in a valid position
[4159] AXT_RT_ERROR_NOT_SAME_MODULE	Not in the same module
[4160] AXT_RT_ERROR_NOT_SAME_BOARD	Not in the same board
[4161] AXT_RT_ERROR_NOT_SAME_PRODUCT	When products are different each other
[4162] AXT_RT_NOT_CAPTURED	Failed to capture the position
[4163] AXT_RT_ERROR_NOT_SAME_IC	Not in the same chip
[4164] AXT_RT_ERROR_NOT_GEARMODE	Failed to change to the gear mode
[4165] AXT_ERROR_CONTI_INVALID_AXIS_NO	Invalid axis during continuous interpolation axix mapping
[4166] AXT_ERROR_CONTI_INVALID_MAP_NO	Invalid mapping number during continuous interpolation mapping
[4167] AXT_ERROR_CONTI_EMPTY_MAP_NO	Continuous interpolation mapping number is empty
[4168] AXT_RT_MOTION_ERROR_CACULATION	Error in calculation
[4169] AXT_RT_ERROR_MOVE_SENSOR_CHECK	Continuous interpolation error sensor (Alarm, EMG, Limit, etc) is detected
[4170] AXT_ERROR_HELICAL_INVALID_AXIS_NO	Invalid axis for helical axis mapping
[4171] AXT_ERROR_HELICAL_INVALID_MAP_NO	Invalid mapping number for helical mapping
[4172] AXT_ERROR_HELICAL_EMPTY_MAP_NO	Helical mapping number is empty
[4180] AXT_ERROR_SPLINE_INVALID_AXIS_NO	Invalid axis for spline axis mapping

[4181] AXT_ERROR_SPLINE_INVALID_MAP_NO	Invalid mapping number for spline mapping
[4182] AXT_ERROR_SPLINE_EMPTY_MAP_NO	Spline Mapping number is empty
[4183] AXT_ERROR_SPLINE_NUM_ERROR	Spline point value is inapplicable
[4184] AXT_RT_MOTION_INTERPOL_VALUE	Invalid input value for interpolation
[4185] AXT_RT_ERROR_NOT_CONTIBEGIN	CONTIBEGIN function is not called in continuous interpolation
[4186] AXT_RT_ERROR_NOT_CONTIEND	CONTIEND function is not called in continuous interpolation
[4201] AXT_RT_MOTION_HOME_SEARCHING	Other function is called while in home search motion
[4202] AXT_RT_MOTION_HOME_ERROR_SEARCHING	Home search motion is fored to stop by the user or something from external
[4203] AXT_RT_MOTION_HOME_ERROR_START	Failed to start home search drive due to intialization problem
[4204] AXT_RT_MOTION_HOME_ERROR_GANTRY	Failed to execute Gantry enable during home search motion
[4251] AXT_RT_MOTION_POS_OUTOFBOUND	Configured position is either above the maximum value or below the minimum value
[4252] AXT_RT_MOTION_PROFILE_INVALID	Invalid setting for velocity profile
[4253] AXT_RT_MOTION_VELOCITY_OUTOFBOUND	Configured velocity is either above the maximum value or below the minimum value
[4254] AXT_RT_MOTION_MOVE_UNIT_IS_ZERO	Unit of the motion values is set as zero
[4255] AXT_RT_MOTION_SETTING_ERROR	Invalid setting for Velocity, Acceleration, Jerk, or Profile
[4256] AXT_RT_MOTION_IN_CONT_INTERPOL	Execuation of motion start or restart function before the completion of current continous interpolated motion
[4257] AXT_RT_MOTION_DISABLE_TRIGGER	Trigger output is disabled
[4258] AXT_RT_MOTION_INVALID_CONT_INDEX	Invalid setting for continuous interpolation index value
[4259] AXT_RT_MOTION_CONT_QUEUE_FULL	Continuous Interpolation queue of motion chip is full
[4260] AXT_RT_PROTECTED_DURING_SERVOON	Can not be used the function or input value in servo-on status
[4261] AXT_RT_HW_ACCESS_ERROR	H/w of board access error has occurred. Check the status of the board mounting system or battery operation to check the status of power input

The contents of this manual can be changed without notice. The name of company, institution, production, people and event used in the examples are not real data. None of actual company, institution, product, people, or events are intended to be connected, and it shall not be interpreted that way. It is user's responsibility to comply with the corresponding copyright law. Separately from the right in the copyright, any part of this manual shall not be copied in any form or means (electronic, mechanical, paper copy, disk copy, or other method), shall not be coped for other purpose, and shall not be stored, introduced, or transmitted without explicit written approval of Ajinextek Co., Ltd.

Ajinextek Co., Ltd reserves all rights including patents, trademark right, copyright, or other intellectual property associated with the contents of this manual. Except the right explicitly given to you from Ajinextek Co., Ltd. by the written end user license agreement, providing this manual does not allow you to use any of patents, copyright, or other intellectual properties.